

Сокеты

```
int socket(int family, int type, int protocol)
// family: AF_UNIX, AF_INET, AF_INET6, AF_IPX, AF_UNSPEC
// type: SOCK_STREAM, SOCK_DGRAM, SOCK_RAW
// protocol: TCP,UDP
```

Листинг 0 — Сокеты

struct socket

```
struct socket {
    socket_state state; /* SS_FREE, SS_UNCONNECTING, SS_CONNECTING,
SS_CONNECTED, SS_DISCONNECTING */

    short type;

    unsigned long flags;

    struct file *file;
    struct sock *sk;
    const struct proto_ops *ops; /* Might change with IPV6_ADDRFORM or
MPTCP. */

    struct socket_wq wq;
};

enum sock_type {
    SOCK_STREAM = 1, /* @SOCK_STREAM: stream (connection) socket */
    SOCK_DGRAM = 2, /* @SOCK_DGRAM: datagram (conn.less) socket */
    SOCK_RAW = 3, /* @SOCK_RAW: raw socket */
    SOCK_RDM = 4,
    SOCK_SEQPACKET = 5,
    SOCK_DCCP = 6,
    SOCK_PACKET = 10,
};
```

Листинг 1 — struct socket

Сокеты AF_INET, сетевой стек

```
struct sockaddr {
    sa_family_t sa_family; /* address family, AF_xxx */
    char sa_data[14]; /* 14 bytes of protocol address */
};

struct sockaddr_in {
    __kernel_sa_family_t sin_family; /* Address family */
    __be16 sin_port; /* Port number */
    struct in_addr sin_addr; /* Internet address */

    /* Pad to size of `struct sockaddr'. */
    unsigned char __pad[__SOCK_SIZE__ - sizeof(short int) -
sizeof(unsigned short int) - sizeof(struct in_addr)];
};
```

```

struct in_addr {
    __be32  s_addr; //_u32
};

/* htons() --- host to network short */

uint16_t htons(uint16_t hostint16);
uint32_t htonl(uint32_t hostint32);

/* ntohs() --- network to host */

uint16_t ntohs(uint16_t netint16);
uint32_t ntohl(uint32_t netint32);

```

Листинг 2 — Сокеты AF_INET, сетевой стек

sys_socketcall

```

asmlinkage long sys_socketcall(int call, unsigned long *args)
{
    int err;
    if (copy_from_user(a, args, args[call]))
        return -EFAULT;

    a0 = a[0];
    a1 = a[1];

    switch (call)
    {
        case SYS_SOCKET:
            err = sys_socket(a0, a1, a[2]);
            break;
        case SYS_BIND:
            err = sys_bind(a0, (struct sockaddr *) a1, a[2]);
            break;
        case SYS_CONNECT:
            err = sys_connect(a0, (struct sockaddr *) a1, a[2]);
            break;
        ...
    }

    return err;
}

/* client: socket() -> [ bind() ] -> connect() -> write() -> close() */
/* server: socket() -> bind() -> listen() -> accept()->read()->close()*/

int  bind(int sockfd, struct sockaddr *addr, int addrlen)
int  connect(int sockfd, const struct sockaddr *serv_addr, int addrlen)
int  accept(int sockfd, struct sockaddr *addr, int *addrlen)
int  listen(int sockfd, int backlog)

```

Листинг 3 — sys_socketcall

sys_socket

```
asm linkage long sys_socket(int family, int type, int protocol)
{
    int retval;
    struct socket *sock;
    ...
    retval = socket_create(family, type, protocol, &sock);
    ...
    return retval;
}
```

Листинг 4 — sys_socket

Мультиплексор

```
struct epoll_event {
    uint32_t      events; /* Epoll events */
    epoll_data_t  data;   /* User data variable */
};
```

```
union epoll_data {
    void      *ptr;
    int       fd;
    uint32_t  u32;
    uint64_t  u64;
};
```

```
typedef union epoll_data  epoll_data_t;
```

```
int epoll_wait(int n, int epfd, struct epoll_event events[n], int n, int
timeout);
```

/* The epoll_wait() system call waits for events on the epoll(7) instance referred to by the file descriptor epfd. The buffer pointed to by events is used to return information from the ready list about file descriptors in the interest list that have some events available. Up to n are returned by epoll_wait(). The n argument must be greater than zero.

The timeout argument specifies the number of milliseconds that epoll_wait() will block. Time is measured against the CLOCK_MONOTONIC clock.

A call to epoll_wait() will block until either:

- a file descriptor delivers an event;
- the call is interrupted by a signal handler; or
- the timeout expires.

Note that the timeout interval will be rounded up to the system clock granularity, and kernel scheduling delays mean that the blocking interval may overrun by a small amount. Specifying a timeout of -1 causes epoll_wait() to block indefinitely, while specifying a timeout equal to zero causes epoll_wait() to return immediately, even if no events are available.

The struct epoll_event is described in epoll_event(3type).

The data field of each returned `epoll_event` structure contains the same data as was specified in the most recent call to `epoll_ctl(2)` (`EPOLL_CTL_ADD`, `EPOLL_CTL_MOD`) for the corresponding open file descriptor.

The events field is a bit mask that indicates the events that have occurred for the corresponding open file description. See `epoll_ctl(2)` for a list of the bits that may appear in this mask. */

```
int epoll_ctl(int epfd, int op, int fd, struct epoll_event *_Nullable event);
```

/* This system call is used to add, modify, or remove entries in the interest list of the `epoll(7)` instance referred to by the file descriptor `epfd`. It requests that the operation `op` be performed for the target file descriptor, `fd`. Valid values for the `op` argument are:

`EPOLL_CTL_ADD`

Add an entry to the interest list of the `epoll` file descriptor, `epfd`. The entry includes the file descriptor, `fd`, a reference to the corresponding open file description (see `epoll(7)` and `open(2)`), and the settings specified in `event`.

`EPOLL_CTL_MOD`

Change the settings associated with `fd` in the interest list to the new settings specified in `event`.

`EPOLL_CTL_DEL`

Remove (deregister) the target file descriptor `fd` from the interest list. The `event` argument is ignored and can be `NULL` (but see `BUGS` below).

The `event` argument describes the object linked to the file descriptor `fd`. The `struct epoll_event` is described in `epoll_event(3type)`. The `data` member of the `epoll_event` structure specifies data that the kernel should save and then return (via `epoll_wait(2)`) when this file descriptor becomes ready.

The `events` member of the `epoll_event` structure is a bit mask composed by ORing together zero or more event types, returned by `epoll_wait(2)`, and input flags, which affect its behavior, but aren't returned. The available event types are:

`EPOLLIN`

The associated file is available for `read(2)` operations.

`EPOLLOUT`

The associated file is available for `write(2)` operations.

`EPOLLET`

Requests edge-triggered notification for the associated file descriptor. The default behavior for `epoll` is level-triggered. See `epoll(7)` for more detailed information about edge-triggered and level-triggered notification.

*/

struct super_block

```

struct super_block {
    struct list_head    s_list;    /* doubly linked lists in Linux */
    dev_t               s_dev;     /* search index; _not_ kdev_t */
    unsigned char       s_blocksize_bits;
    unsigned long       s_blocksize;
    loff_t              s_maxbytes; /* Max file size */
    struct file_system_type *s_type; /* pointer to FS type
(file_system_type) */
    const struct super_operations *s_op; /* pointer to superblock
operations (super_operations) */
    /* ... */
    unsigned long       s_flags;
    unsigned long       s_iflags; /* internal SB_I_* flags */
    unsigned long       s_magic; /* FS magic number */
    struct dentry       *s_root; /* mount point, root directory of mounted
FS */
    struct rw_semaphore s_umount; /* reader-writer semaphore for
mounting */
    /* ... */
    struct mount        *s_mounts; /* list of mounts of this FS (same FS
can be mounted multiple times) */
    struct block_device *s_bdev; /* associated block device (can go
away once we use an accessor for @s_bdev_file) */
    /* ... */
    struct list_head    s_inodes; /* list of all inodes of this FS */
    /* ... */
    struct list_head    s_inodes_wb; /* writeback inodes */
    /* ... */
}

struct super_operations {
    struct inode *(*alloc_inode)(struct super_block *sb); /* Creates
and initializes a new inode object for a specific mounted FS (allocates
address space) */
    void (*destroy_inode)(struct inode *inode); /* Removes inode from
the linked list of inodes */
    void (*free_inode)(struct inode *inode); /* Metadata (with inode
number) is not deleted; this inode can be reused */
    void (*dirty_inode)(struct inode *inode, int flags); /* Called to
move inode to another list if changes were made to it */
    int (*write_inode)(struct inode *inode, struct writeback_control
*wbc); /* Writes inode to disk (alloc alone is not enough) */
    int (*drop_inode)(struct inode *inode); /* Called by VFS interface
when the inode reference count reaches 0 */

    void (*put_super)(struct super_block *sb); /* Called by VFS when
unmounting FS; this superblock is removed from the mount list */

    int (*remount_fs)(struct super_block *sb, int *flags, char *data); /
* Called by VFS to remount the superblock with different parameters */

    void (*umount_begin)(struct super_block *sb); /* Called by VFS to

```

```

abort mounting; used by network FSes like NFS */
    ...
};

```

Листинг 6 — struct super_block

struct inode

```

struct inode {
    umode_t    i_mode; /* File type and permissions */
    unsigned short i_opflags;
    unsigned int i_flags; /* Permissions and flags; flags act as
restrictions, indicating what can and cannot be done. */
    ...
    const struct inode_operations *i_op; /* Pointer to the inode operations
table (inode_operations). */
    struct super_block *i_sb; /* Pointer to the associated superblock: the
filesystem is mounted on a specific physical device, and any file belongs
to a specific FS. */
    struct address_space *i_mapping;
#ifdef CONFIG_SECURITY
    void *i_security;
#endif
    unsigned long i_ino; /* Inode number. */
    union {
        const unsigned int i_nlink ; /* Number of hard links. */
        unsigned int __i_nlink;
    }; /*
    * Filesystems may only read i_nlink directly. They shall use the
    * following functions for modification:
    *
    * (set|clear|inc|drop)_nlink
    * inode_(inc|dec)_link_count
    */

    dev_t    i_rdev;
    loff_t    i_size; /* File size; since files are associated with block
devices, the inode contains block size information (i_blkbits). */
    ...
    time64_t    i_atime_sec;
    time64_t    i_mtime_sec;
    time64_t    i_ctime_sec;
    u32    i_atime_nsec;
    u32    i_mtime_nsec;
    u32    i_ctime_nsec;
    ...
    u8    i_blkbits;
    ...
    struct rw_semaphore i_rwsem; /* Read-write semaphore. */
    ...
    struct hlist_node i_hash; /* Provides fast access to files (hash list).
*/
    ...
    struct list_head i_lru; /* Provides fast access to files (LRU list).
*/
    struct list_head i_sb_list; /* Provides fast access to files

```

```

(superblock list). */
struct list_head i_wb_list; /* backing dev writeback list */
union {
    struct hlist_head i_dentry;
    struct rcu_head i_rcu;
};
...
atomic_t i_readcount; /* Counter for the number of read accesses to
the file (via struct file). */
union {
    const struct file_operations *i_fop; /* former ->i_op-
>default_file_ops */
    void (*free_inode)(struct inode *);
};
...
union {
    struct pipe_inode_info *i_pipe;
    struct cdev *i_cdev;
    char *i_link;
    unsigned i_dir_seq;
};
}

```

Листинг 7 — struct inode

struct inode_operations

```

struct inode_operations {
    struct dentry * (*lookup) (struct inode *, struct dentry *, unsigned
int);
    const char * (*get_link) (struct dentry *, struct inode *, struct
delayed_call *);
    int (*permission) (struct mnt_idmap *, struct inode *, int);
    struct posix_acl * (*get_inode_acl)(struct inode *, int, bool);

    int (*readlink) (struct dentry *, char __user *, int);

    int (*create) (struct mnt_idmap *, struct inode *, struct dentry *,
umode_t, bool);
    int (*link) (struct dentry *, struct inode *, struct dentry *);
    int (*unlink) (struct inode *, struct dentry *);
    int (*symlink) (struct mnt_idmap *, struct inode *, struct dentry *,
const char *);
    struct dentry * (*mkdir) (struct mnt_idmap *, struct inode *,
struct dentry *, umode_t);
    int (*rmdir) (struct inode *, struct dentry *);
    int (*mknod) (struct mnt_idmap *, struct inode *, struct dentry *,
umode_t, dev_t);
    int (*rename) (struct mnt_idmap *, struct inode *, struct dentry *,
struct inode *, struct dentry *, unsigned int);
    int (*setattr) (struct mnt_idmap *, struct dentry *, struct iattr *);
    int (*getattr) (struct mnt_idmap *, const struct path *,
struct kstat *, u32, unsigned int);
    ssize_t (*listxattr) (struct dentry *, char *, size_t);
    int (*fiemap)(struct inode *, struct fiemap_extent_info *, u64 start,

```

```

        u64 len);
int (*update_time)(struct inode *inode, enum fs_update_time type,
    unsigned int flags);
void (*sync_lazytime)(struct inode *inode);
int (*atomic_open)(struct inode *, struct dentry *,
    struct file *, unsigned open_flag,
    umode_t create_mode);
int (*tmpfile)(struct mnt_idmap *, struct inode *,
    struct file *, umode_t);
struct posix_acl *(*get_acl)(struct mnt_idmap *, struct dentry *,
    int);
int (*set_acl)(struct mnt_idmap *, struct dentry *,
    struct posix_acl *, int);
int (*fileattr_set)(struct mnt_idmap *idmap,
    struct dentry *dentry, struct file_kattr *fa);
int (*fileattr_get)(struct dentry *dentry, struct file_kattr *fa);
struct offset_ctx *(*get_offset_ctx)(struct inode *inode);
}

```

Листинг 8 — struct inode_operations

struct dentry

```

struct dentry {
    /* RCU lookup touched fields */
    unsigned int d_flags; /* protected by d_lock */
    /* ... */
    struct hlist_bl_node d_hash; /* lookup hash list */
    struct dentry *d_parent; /* parent directory */
    struct qstr d_name;
    struct inode *d_inode; /* Where the name belongs to - NULL is
negative */
    /* ... */
    const struct dentry_operations *d_op; /* per-dentry operations listed
for each path element */
    struct super_block *d_sb; /* The root of the dentry tree - only for
mounted FS, access to its dirs/files; any dir/file belongs to a specific
FS */
    /* ... */
    struct list_head d_lru; /* LRU list */
    /* ... */
    struct list_head d_subdirs; /* our children */
    /* ... */
};

```

Листинг 9 — struct dentry

Функции для работы с кэшем SLAB

```

struct kmem_cache *kmem_cache_create(const char *name, size_t size,
size_t offset,
                                unsigned long flags, void
(*constructor)(void *));

void kmem_cache_destroy(struct kmem_cache *cache);
void *kmem_cache_alloc(struct kmem_cache *cache, gfp_t flags);

```

```
void kmem_cache_free(struct kmem_cache *cache, void *obj);
}
```

Листинг 10 — Функции для работы с кэшем SLAB

struct file_system_type

```
struct file_system_type {
    const char *name; /* File system name. */
    int fs_flags; /* File system type flags;
FS_REQUIRES_DEV is needed for FS that works with external storage (affects
mount functions). FS_USERNS_MOUNT means not everything must be done in
kernel. */
    struct dentry *(*mount) (struct file_system_type *, int, const char *,
void *); /* Mount function. */
    void (*kill_sb) (struct super_block *); /* Unmount (kill superblock)
function. */
    struct module *owner; /* Owner module; needed for
reference counting (module cannot be unloaded while FS is mounted). */
    struct file_system_type *next; /* Linked list of
file_system_type. */
    struct hlist_head fs_supers; /* List of superblocks. */
};

extern int register_filesystem(struct file_system_type *);
extern int unregister_filesystem(struct file_system_type *);

extern struct dentry *mount_bdev(struct file_system_type *fs_type, int
flags,
    const char *dev_name, void *data, int (*fill_super)
(struct super_block *, void *, int));

extern struct dentry *mount_nodev(struct file_system_type *fs_type, int
flags, void *data, int (*fill_super) (struct super_block *, void *, int));

/* Mount function parameters:
* fs_type - filesystem type
* flags - mount flags
* dev_name - device name (only for mount_bdev; the external difference
between the two functions is this parameter)
* data - arbitrary mount data
* fill_super - pointer to the superblock filling function
*/
/* The functions differ significantly in code and scope of application:
* mount_bdev - for mounting a filesystem located on a block device
* mount_nodev - for mounting a filesystem not associated with any device
*/
struct vfsmount {
    struct dentry *mnt_root; /* root of the mounted tree */
    struct super_block *mnt_sb; /* pointer to superblock */
    int mnt_flags;
    struct mnt_idmap *mnt_idmap;
};
```

Листинг 11 — struct file_system_type

simple_fill_super

```
int simple_fill_super(struct super_block *s, unsigned long magic,
                     const struct tree_descr *files)
{
    struct inode *inode;
    struct dentry *dentry;
    int i;

    s->s_blocksize = PAGE_SIZE;
    s->s_blocksize_bits = PAGE_SHIFT;
    s->s_magic = magic;
    s->s_op = &simple_super_operations;
    s->s_time_gran = 1;

    inode = new_inode(s);
    if (!inode)
        return -ENOMEM;
    /*
     * because the root inode is 1, the files array must not contain an
     * entry at index 1
     */
    inode->i_ino = 1;
    inode->i_mode = S_IFDIR | 0755;
    simple_inode_init_ts(inode);
    inode->i_op = &simple_dir_inode_operations;
    inode->i_fop = &simple_dir_operations;
    set_nlink(inode, 2);
    s->s_root = d_make_root(inode);
    if (!s->s_root)
        return -ENOMEM;
    for (i = 0; !files->name || files->name[0]; i++, files++) {
        if (!files->name)
            continue;

        /* warn if it tries to conflict with the root inode */
        if (unlikely(i == 1))
            printk(KERN_WARNING "%s: %s passed in a files array"
                    "with an index of 1!\n", __func__,
                    s->s_type->name);

        dentry = d_alloc_name(s->s_root, files->name);
        if (!dentry)
            return -ENOMEM;
        inode = new_inode(s);
        if (!inode) {
            dput(dentry);
            return -ENOMEM;
        }
        inode->i_mode = S_IFREG | files->mode;
        simple_inode_init_ts(inode);
        inode->i_fop = files->ops;
        inode->i_ino = i;
        d_make_persistent(dentry, inode);
        dput(dentry);
    }
}
```

```

    return 0;
}

struct inode *new_inode(struct super_block *sb)
{
    struct inode *inode = new_inode_pseudo(sb); // -> alloc_inode(sb): from
slab
    if (inode)
        inode_sb_list_add(inode);
    return inode;
}

```

Листинг 12 — simple_fill_super

struct proc_dir_entry

```

struct proc_dir_entry {
    atomic_t in_use;                                /* atomic counter */
    ...
    struct list_head pde_openers;
    ...
    const struct inode_operations *proc_iops; /* inode operations for proc
file */
    union {
        const struct proc_ops *proc_ops;
        const struct file_operations *proc_dir_ops;
    }; /* function registration interface (proc_ops introduced to offload
overloaded file_operations) */
    ...
    const struct dentry_operations *proc_dops;
    union {
        const struct seq_operations *seq_ops;
        int (*single_show)(struct seq_file *, void *);
    }; /* seq interfaces: full (seq_ops) or simplified (single_show) */
    ...
    proc_write_t write; /* write callback (read removed, write remains
for user->kernel) */
    void *data; /* private data */
    unsigned int state_size;
    unsigned int low_ino; /* low-level inode (virtual FS, inode exists in
RAM only) */
    nlink_t nlink;
    ...
    loff_t size; /* file size (loff_t — typed for compile-time checking) */
    struct proc_dir_entry *parent; /* parent directory (maintains proc
hierarchy) */
    ...
    char *name;
    ...
    u8 namelen;
    char inline_name[];
};

```

Листинг 13 — struct proc_dir_entry

struct proc_ops

```
extern struct proc_dir_entry *proc_create_data(const char *, umode_t,
struct proc_dir_entry *, const struct proc_ops *, void *);

struct proc_dir_entry *proc_create(const char *name, umode_t mode, struct
proc_dir_entry *parent, const struct proc_ops *proc_ops);

extern struct proc_dir_entry *proc_mkdir(const char *, struct
proc_dir_entry *);

extern struct proc_dir_entry *proc_symlink(const char *, struct
proc_dir_entry *, const char *);

extern void remove_proc_entry(const char *, struct proc_dir_entry *);

struct proc_ops {
    unsigned int proc_flags;
    int (*proc_open)(struct inode *, struct file *);
    ssize_t (*proc_read)(struct file *, char __user *, size_t, loff_t *);
    ssize_t (*proc_read_iter)(struct kiocb *, struct iov_iter *);
    ssize_t (*proc_write)(struct file *, const char __user *, size_t, loff_t
*);
    /* mandatory unless nonseekable_open() or equivalent is used */
    loff_t (*proc_lseek)(struct file *, loff_t, int);
    int (*proc_release)(struct inode *, struct file *);
    __poll_t (*proc_poll)(struct file *, struct poll_table_struct *);
    long (*proc_ioctl)(struct file *, unsigned int, unsigned long);
#ifdef CONFIG_COMPAT
    long (*proc_compat_ioctl)(struct file *, unsigned int, unsigned long);
#endif
    int (*proc_mmap)(struct file *, struct vm_area_struct *);
    unsigned long (*proc_get_unmapped_area)(struct file *, unsigned long,
unsigned long, unsigned long, unsigned long);
} __randomize_layout;

static const struct proc_ops device_proc_ops = {
    .proc_open = device_open,
    .proc_read = device_read,
    .proc_write = device_write,
    .proc_release = device_release,
};

unsigned long __copy_to_user(void __user *to, const void *from, unsigned
long n);
unsigned long __copy_from_user(void *to, const void __user *from, unsigned
long n);
```

Листинг 14 — struct proc_ops

struct file

```
struct file {
    ...
    spinlock_t          f_lock;          /* Exclusive access to the file.
*/
```

```

    fmode_t          f_mode;          /* Access mode. */
    atomic_long_t    f_count;         /* File reference counter. */
    loff_t           f_pos;           /* Logical position in the file:
file is assumed to start at offset 0; each read/write operation changes
this field by the corresponding number of bytes. */

    ...
    struct path      f_path;          /* Link to dentry and mounted file
system. */
    struct inode      *f_inode;        /* Pointer to the inode (physical
file). */
    const struct file_operations *f_op; /* Operations defined on an open
file. */
    ...
};

```

Листинг 15 — struct file

struct file_operations

```

struct file_operations {
    struct module *owner;
    fop_flags_t fop_flags;
    loff_t (*llseek) (struct file *, loff_t, int);
    ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);
    ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);
    ssize_t (*read_iter) (struct kiocb *, struct iov_iter *);
    ssize_t (*write_iter) (struct kiocb *, struct iov_iter *);
    int (*iopoll)(struct kiocb *kiocb, struct io_comp_batch *,
        unsigned int flags);
    int (*iterate_shared) (struct file *, struct dir_context *);
    __poll_t (*poll) (struct file *, struct poll_table_struct *);
    long (*unlocked_ioctl) (struct file *, unsigned int, unsigned long);
    long (*compat_ioctl) (struct file *, unsigned int, unsigned long);
    int (*mmap) (struct file *, struct vm_area_struct *);
    int (*open) (struct inode *, struct file *);
    int (*flush) (struct file *, fl_owner_t id);
    int (*release) (struct inode *, struct file *);
    int (*fsync) (struct file *, loff_t, loff_t, int datasync);
    int (*fasync) (int, struct file *, int);
    int (*lock) (struct file *, int, struct file_lock *);
    unsigned long (*get_unmapped_area)(struct file *, unsigned long,
unsigned long, unsigned long, unsigned long);
    int (*check_flags)(int);
    int (*flock) (struct file *, int, struct file_lock *);
    ssize_t (*splice_write)(struct pipe_inode_info *, struct file *, loff_t
*, size_t, unsigned int);
    ssize_t (*splice_read)(struct file *, loff_t *, struct pipe_inode_info
*, size_t, unsigned int);
    void (*splice_eof)(struct file *file);
    int (*setlease)(struct file *, int, struct file_lease **, void **);
    long (*fallocate)(struct file *file, int mode, loff_t offset,
        loff_t len);
    void (*show_fdinfo)(struct seq_file *m, struct file *f);
#ifdef CONFIG_MMU
    unsigned (*mmap_capabilities)(struct file *);
#endif
};

```

```

ssize_t (*copy_file_range)(struct file *, loff_t, struct file *,
    loff_t, size_t, unsigned int);
loff_t (*remap_file_range)(struct file *file_in, loff_t pos_in,
    struct file *file_out, loff_t pos_out,
    loff_t len, unsigned int remap_flags);
int (*fadvise)(struct file *, loff_t, loff_t, int);
int (*uring_cmd)(struct io_uring_cmd *ioucmd, unsigned int issue_flags);
int (*uring_cmd_iopoll)(struct io_uring_cmd *, struct io_comp_batch *,
    unsigned int poll_flags);
int (*mmap_prepare)(struct vm_area_desc *);
}

```

Листинг 16 — struct file_operations

open

```

int open(const char *pathname, int flags); /* pathname — Name of the file
to open. flags — Flags defining access mode and behavior. */
int open(const char *pathname, int flags, mode_t mode); /* mode — Access
permissions for the created file. The third parameter is required only
when open() is called to create a new file (with O_CREAT). If the file
already exists, mode is ignored. */

/* Returns a file descriptor — a non-negative integer, which is the lowest
 * unused descriptor in the process's open file table. Upon creation of a
new file, a new inode is initialized. When a file is opened, the struct
file fields (including f_pos) are initialized, with the file offset set to
the beginning of the file.
 */

/*
 * Access flags (one of them must be specified):
 *   O_RDONLY — Open file for reading only.
 *   O_WRONLY — Open file for writing only.
 *   O_RDWR  — Open file for reading and writing.
 *
 * Creation and behavior flags:
 *   O_CREAT   — Create the file if it does not exist. When this flag
is used, the mode parameter must be provided to set access permissions.
 *   O_APPEND  — Append to the existing file: before each write() call,
the file offset is moved to the end, so previously written data is
not overwritten.
 *   O_TRUNC   — If the file exists and is opened for writing, truncate
it to zero length.
 *   O_NONBLOCK — Set non-blocking mode. Useful for special files like
FIFOs and terminals; open() returns immediately instead of blocking.
Also affects read/write operations, allowing them to return
immediately with an error if no input is available or output cannot
be written.
 *   O_CLOEXEC  — Set the close-on-exec flag on the new file descriptor.
 *   O_PATH     — Obtain only a file descriptor (the file itself is not
actually opened for reading/writing).
 *   O_TMPFILE  — Create an unnamed temporary regular file.
 *
 * MODE (permissions for O_CREAT):
 * When creating a new file, the owner and group IDs are taken from the

```

```

* process's effective IDs. The following symbolic constants are
available:
*   S_IRWXU / S_IRWXG / S_IRWXO – Read, write, and execute permissions
    for owner / group / others.
*   S_IRUSR / S_IRGRP / S_IROTH – Read permission.
*   S_IWUSR / S_IWGRP / S_IWOTH – Write permission.
*   S_IXUSR / S_IXGRP / S_IXOTH – Execute permission.
*/

```

Листинг 17 – open

struct FILE

```

typedef struct _IO_FILE FILE;

FILE *fopen(const char *fname, const char *mode);

```

Листинг 18 – struct FILE

struct _IO_FILE

```

struct _IO_FILE {
    int _flags; /* High-order word is _IO_MAGIC; rest is flags. */
#define _IO_file_flags _flags

    /* The following pointers correspond to the C++ streambuf protocol. */
    /* Note: Tk uses the _IO_read_ptr and _IO_read_end fields directly. */
    char* _IO_read_ptr; /* Current read pointer */
    char* _IO_read_end; /* End of get area. */
    char* _IO_read_base; /* Start of putback+get area. */
    char* _IO_write_base; /* Start of put area. */
    char* _IO_write_ptr; /* Current put pointer. */
    char* _IO_write_end; /* End of put area. */
    char* _IO_buf_base; /* Start of reserve area. */
    char* _IO_buf_end; /* End of reserve area. */
    /* The following fields are used to support backing up and undo. */
    char *_IO_save_base; /* Pointer to start of non-current get area. */
    char *_IO_backup_base; /* Pointer to first valid character of backup
area */
    char *_IO_save_end; /* Pointer to end of non-current get area. */

    struct _IO_marker *_markers;
    struct _IO_FILE *_chain;
    int _fileno;
#if 0
    int _blksize;
#else
    int _flags2;
#endif
    _IO_off_t _old_offset; /* This used to be _offset but it's too small.
*/

#define __HAVE_COLUMN /* temporary */
    /* 1+column number of pbase(); 0 is unknown. */
    unsigned short _cur_column;
    signed char _vtable_offset;

```

```

char _shortbuf[1];

/* char* _save_gptr; char* _save_egptr; */

_IO_lock_t *_lock;
#ifdef _IO_USE_OLD_IO_FILE
};

```

Листинг 19 — struct IO_FILE

struct stat

```

struct stat {
    dev_t      st_dev;          /* [XSI] ID of device containing file */
    ino_t      st_ino;          /* [XSI] File serial number */
    mode_t     st_mode;         /* [XSI] Mode of file (see below) */
    nlink_t    st_nlink;        /* [XSI] Number of hard links */
    uid_t      st_uid;          /* [XSI] User ID of the file */
    gid_t      st_gid;          /* [XSI] Group ID of the file */
    dev_t      st_rdev;         /* [XSI] Device ID */
#ifdef _POSIX_C_SOURCE || defined(_DARWIN_C_SOURCE)
    struct timespec st_atimespec; /* time of last access */
    struct timespec st_mtimespec; /* time of last data modification */
    struct timespec st_ctimespec; /* time of last status change */
#else
    time_t      st_atime;        /* [XSI] Time of last access */
    long        st_atimensec;    /* nsec of last access */
    time_t      st_mtime;        /* [XSI] Last data modification time */
    long        st_mtimensec;    /* last data modification nsec */
    time_t      st_ctime;        /* [XSI] Time of last status change */
    long        st_ctimensec;    /* nsec of last status change */
#endif
    off_t      st_size;         /* [XSI] file size, in bytes */
    blkcnt_t   st_blocks;       /* [XSI] blocks allocated for file */
    blksize_t   st_blksize;     /* [XSI] optimal blocksize for I/O */
    __uint32_t  st_flags;        /* user defined flags for file */
    __uint32_t  st_gen;          /* file generation number */
    __int32_t   st_lspare;       /* RESERVED: DO NOT USE! */
    __int64_t   st_qspare[2];    /* RESERVED: DO NOT USE! */
};

```

Листинг 20 — struct stat

Обработчики аппаратных прерываний

```

/**
 * enum irqreturn - irqreturn type values
 * @IRQ_NONE:   interrupt was not from this device or was not handled
 * @IRQ_HANDLED: interrupt was handled by this device
 * @IRQ_WAKE_THREAD: handler requests to wake the handler thread
 */
enum irqreturn {
    IRQ_NONE      = (0 << 0),
    IRQ_HANDLED   = (1 << 0),
    IRQ_WAKE_THREAD = (1 << 1),
};

```

```
typedef enum irqreturn_t;
}
```

Листинг 21 — Обработчики аппаратных прерываний

Флаги IRQ

```
/*
 * IRQF_SHARED - allow sharing the irq among several devices
 * IRQF_PROBE_SHARED - set by callers when they expect sharing mismatches
to occur
 * IRQF_TIMER - Flag to mark this interrupt as timer interrupt
 * IRQF_PERCPU - Interrupt is per cpu
 * IRQF_NOBALANCING - Flag to exclude this interrupt from irq balancing
 * IRQF_IRQPOLL - Interrupt is used for polling (only the interrupt that
is
 *
    registered first in a shared interrupt is considered for
 *
    performance reasons)
 * IRQF_ONESHOT - Interrupt is not reenabled after the hardirq handler
finished.
 *
    Used by threaded interrupts which need to keep the
 *
    irq line disabled until the threaded handler has been
run.
 * IRQF_NO_SUSPEND - Do not disable this IRQ during suspend. Does not
guarantee
 *
    that this interrupt will wake the system from a
suspended
 *
    state. See Documentation/power/suspend-and-
interrupts.rst
 * IRQF_FORCE_RESUME - Force enable it on resume even if IRQF_NO_SUSPEND
is set
 * IRQF_NO_THREAD - Interrupt cannot be threaded
 * IRQF_EARLY_RESUME - Resume IRQ early during syscore instead of at
device
 *
    resume time.
 * IRQF_COND_SUSPEND - If the IRQ is shared with a NO_SUSPEND user,
execute this
 *
    interrupt handler after suspending interrupts. For
system
 *
    wakeup devices users need to implement wakeup detection
in
 *
    their interrupt handlers.
 * IRQF_NO_AUTOEN - Don't enable IRQ or NMI automatically when users
request it.
 *
    Users will enable it explicitly by enable_irq() or
enable_nmi()
 *
    later.
 * IRQF_NO_DEBUG - Exclude from runaway detection for IPI and similar
handlers,
 *
    depends on IRQF_PERCPU.
 * IRQF_COND_ONESHOT - Agree to do IRQF_ONESHOT if already set for a
shared
 *
    interrupt.
 */
#define IRQF_SHARED 0x00000080
```

```

#define IRQF_PROBE_SHARED 0x00000100
#define __IRQF_TIMER      0x00000200
#define IRQF_PERCPU       0x00000400
#define IRQF_NOBALANCING  0x00000800
#define IRQF_IRQPOLL      0x00001000
#define IRQF_ONESHOT      0x00002000
#define IRQF_NO_SUSPEND   0x00004000
#define IRQF_FORCE_RESUME 0x00008000
#define IRQF_NO_THREAD    0x00010000
#define IRQF_EARLY_RESUME 0x00020000
#define IRQF_COND_SUSPEND 0x00040000
#define IRQF_NO_AUTOEN    0x00080000
#define IRQF_NO_DEBUG     0x00100000
#define IRQF_COND_ONESHOT 0x00200000

```

Листинг 22 — Флаги IRQ

struct irqaction

```

/**
 * struct irqaction - per interrupt action descriptor
 * @handler: interrupt handler function
 * @name: name of the device
 * @dev_id: cookie to identify the device
 * @percpu_dev_id: cookie to identify the device
 * @affinity: CPUs this irqaction is allowed to run on
 * @next: pointer to the next irqaction for shared interrupts
 * @irq: interrupt number
 * @flags: flags (see IRQF_* above)
 * @thread_fn: interrupt handler function for threaded interrupts
 * @thread: thread pointer for threaded interrupts
 * @secondary: pointer to secondary irqaction (force threading)
 * @thread_flags: flags related to @thread
 * @thread_mask: bitmask for keeping track of @thread activity
 * @dir: pointer to the proc/irq/NN/name entry
 */
struct irqaction {
    irq_handler_t handler;
    union {
        void *dev_id;
        void __percpu *percpu_dev_id;
    };
    const struct cpumask *affinity;
    struct irqaction *next;
    irq_handler_t thread_fn;
    struct task_struct *thread;
    struct irqaction *secondary;
    unsigned int irq;
    unsigned int flags;
    unsigned long thread_flags;
    unsigned long thread_mask;
    const char *name;
    struct proc_dir_entry *dir;
}

/**

```

```

* request_irq - Add a handler for an interrupt line
* @irq: The interrupt line to allocate
* @handler: Function to be called when the IRQ occurs.
*   Primary handler for threaded interrupts
*   If NULL, the default primary handler is installed
* @flags: Handling flags
* @name: Name of the device generating this interrupt
* @dev: A cookie passed to the handler function
*
* This call allocates an interrupt and establishes a handler; see
* the documentation for request_threaded_irq() for details.
*/

static inline int __must_check
request_irq(unsigned int irq, irq_handler_t handler, unsigned long flags,
            const char *name, void *dev)
{
    return request_threaded_irq(irq, handler, NULL, flags |
    IRQF_COND_ONESHOT, name, dev);
}

extern const void *free_irq(unsigned int, void *); /* free irq line from
handler */

```

Листинг 23 — struct irqaction

softirq

```

/* PLEASE, avoid to allocate new softirqs, if you need not _really_ high
frequency threaded job scheduling. For almost all the purposes
tasklets are more than enough. F.e. all serial device BHs et
al. should be converted to tasklets, not to softirqs.
*/

enum
{
    HI_SOFTIRQ=0,
    TIMER_SOFTIRQ,
    NET_TX_SOFTIRQ, /* network */
    NET_RX_SOFTIRQ, /* network */
    BLOCK_SOFTIRQ,
    IRQ_POLL_SOFTIRQ,
    TASKLET_SOFTIRQ,
    SCHED_SOFTIRQ,
    HRTIMER_SOFTIRQ,
    RCU_SOFTIRQ, /* Preferable RCU should always be the last softirq */

    NR_SOFTIRQS
};

struct softirq_action {
    void (*action)(struct softirq_action *); /* executable function */
};

static struct softirq_action softirq_vec[NR_SOFTIRQS];

```

```

const char * const softirq_to_name[NR_SOFTIRQS] = {
    "HI", "TIMER", "NET_TX", "NET_RX", "BLOCK", "IRQ_POLL", "TASKLET",
    "SCHED", "HRTIMER", "RCU"
};

void raise_softirq(unsigned int nr); /* Marks softirq as pending on
current CPU (called from HW interrupt handler) */

void open_softirq(int nr, void (*action)(void)); /* Registers softirq
handler (called during initialization) */

/*
 * open_softirq - registers handler (nr = index in softirq_vec, action =
handler function)
 * raise_softirq - marks registered softirq as pending so it can be
executed
 */

```

Листинг 24 — softirq

tasklet

```

/* Tasklets --- multithreaded analogue of BHs.
Main feature differing them of generic softirqs: tasklet
is running only on one CPU simultaneously.

Main feature differing them of BHs: different tasklets
may be run simultaneously on different CPUs.

Properties:
 * If tasklet_schedule() is called, then tasklet is guaranteed
to be executed on some cpu at least once after this.
 * If the tasklet is already scheduled, but its execution is still not
started, it will be executed only once.
 * If this tasklet is already running on another CPU (or schedule is
called
from tasklet itself), it is rescheduled for later.
 * Tasklet is strictly serialized wrt itself, but not
wrt another tasklets. If client needs some intertask synchronization,
he makes it with spinlocks.
 */

struct tasklet_struct
{
    struct tasklet_struct *next;
    unsigned long state;
    atomic_t count;
    bool use_callback;
    union {
        void (*func)(unsigned long data);
        void (*callback)(struct tasklet_struct *t);
    };
    unsigned long data;
};

```

```

enum
{
    TASKLET_STATE_SCHED, /* Tasklet is scheduled for execution */
    TASKLET_STATE_RUN /* Tasklet is running (SMP only) */
};

#define DECLARE_TASKLET(name, _callback) \
struct tasklet_struct name = { \
    .count = ATOMIC_INIT(0), \
    .callback = _callback, \
    .use_callback = true, \
}

#define DECLARE_TASKLET_DISABLED(name, _callback) \
struct tasklet_struct name = { \
    .count = ATOMIC_INIT(1), \
    .callback = _callback, \
    .use_callback = true, \
}

#define from_tasklet(var, callback_tasklet, tasklet_fieldname) \
    container_of(callback_tasklet, typeof(*var), tasklet_fieldname)

void tasklet_init(struct tasklet_struct *t,
    void (*func)(unsigned long), unsigned long data)
{
    t->next = NULL;
    t->state = 0;
    atomic_set(&t->count, 0);
    t->func = func;
    t->use_callback = false;
    t->data = data;
}

static void __tasklet_schedule_common(struct tasklet_struct *t,
    struct tasklet_head __percpu *headp,
    unsigned int softirq_nr)
{
    struct tasklet_head *head;
    unsigned long flags;

    local_irq_save(flags);
    head = this_cpu_ptr(headp);
    t->next = NULL;
    *head->tail = t;
    head->tail = &(t->next);
    raise_softirq_irqoff(softirq_nr);
    local_irq_restore(flags);
}

void __tasklet_schedule(struct tasklet_struct *t)
{
    __tasklet_schedule_common(t, &tasklet_vec,
        TASKLET_SOFTIRQ);
}

```

```
EXPORT_SYMBOL(__tasklet_schedule);

void __tasklet_hi_schedule(struct tasklet_struct *t)
{
    __tasklet_schedule_common(t, &tasklet_hi_vec,
                              HI_SOFTIRQ);
}
EXPORT_SYMBOL(__tasklet_hi_schedule);
```

Листинг 25 — tasklet

struct worker

```
struct worker_pool;
/*
 * The poor guys doing the actual heavy lifting. All on-duty workers are
 * either serving the manager role, on idle list or on busy hash. For
 * details on the locking annotation (L, I, X...), refer to workqueue.c.
 * Only to be used in workqueue and async.
 */
struct worker {
    /* on idle list while idle, on busy hash table while busy */
    union {
        struct list_head entry; /* L: while idle */
        struct hlist_node hentry; /* L: while busy */
    };

    struct work_struct *current_work; /* K: work being processed and its
 */
    work_func_t current_func; /* K: function */
    struct pool_workqueue *current_pwq; /* K: pwq */
    u64 current_at; /* K: runtime at start or last wakeup */
    unsigned long current_start; /* K: start time of current work item */
    unsigned int current_color; /* K: color */

    int sleeping; /* S: is worker sleeping? */

    /* used by the scheduler to determine a worker's last known identity */
    work_func_t last_func; /* K: last work's fn */

    struct list_head scheduled; /* L: scheduled works */

    struct task_struct *task; /* I: worker task */
    struct worker_pool *pool; /* A: the associated pool */
    /* L: for rescuers */
    struct list_head node; /* A: anchored at pool->workers */
    /* A: runs through worker->node */

    unsigned long last_active; /* K: last active timestamp */
    unsigned int flags; /* L: flags */
    int id; /* I: worker id */

    /*
     * Opaque string set with work_set_desc(). Printed out with task
     * dump for debugging - WARN, BUG, panic or sysrq.
     */
};
```

```

char        desc[WORKER_DESC_LEN];

/* used only by rescuers to point to the target workqueue */
struct workqueue_struct *rescue_wq; /* I: the workqueue to rescue */
};

```

Листинг 26 — struct worker

struct worker_pool

```

struct worker_pool {
    raw_spinlock_t    lock;    /* the pool lock */
    int               cpu;    /* I: the associated cpu */
    int               node;    /* I: the associated node ID */
    int               id;    /* I: pool ID */
    unsigned int      flags;    /* L: flags */

    unsigned long     last_progress_ts; /* L: last forward progress timestamp */
    bool              cpu_stall; /* WD: stalled cpu bound pool */
    /*
     * The counter is incremented in a process context on the associated CPU
     * w/ preemption disabled, and decremented or reset in the same context
     * but w/ pool->lock held. The readers grab pool->lock and are
     * guaranteed to see if the counter reached zero.
     */
    int               nr_running;

    struct list_head   worklist; /* L: list of pending works */

    int               nr_workers; /* L: total number of workers */
    int               nr_idle; /* L: currently idle workers */

    struct list_head   idle_list; /* L: list of idle workers */
    struct timer_list  idle_timer; /* L: worker idle timeout */
    struct work_struct  idle_cull_work; /* L: worker idle cleanup */

    struct timer_list  mayday_timer; /* L: SOS timer for workers */

    /* a workers is either on busy_hash or idle_list, or the manager */
    DECLARE_HASHTABLE(busy_hash, BUSY_WORKER_HASH_ORDER);
    /* L: hash of busy workers */

    struct worker      *manager; /* L: purely informational */
    struct list_head    workers; /* A: attached workers */

    struct ida          worker_ida; /* worker IDs for task name */

    struct workqueue_attrs *attrs; /* I: worker attributes */
    struct hlist_node   hash_node; /* PL: unbound_pool_hash node */
    int                 refcnt; /* PL: refcnt for unbound pools */
#ifdef CONFIG_PREEMPT_RT
    spinlock_t          cb_lock; /* BH worker cancel lock */
#endif
    /*
     * Destruction of pool is RCU protected to allow dereferences
     */
};

```

```

    * from get_work_pool().
    */
    struct rcu_head    rcu;
};

```

Листинг 27 — struct worker_pool

Инициализация очереди работ

```

#define DECLARE_WORK(n, f) \
    struct work_struct n = __WORK_INITIALIZER(n, f)

#define __INIT_WORK(_work, _func, _onstack) \
    do { \
        static __maybe_unused struct lock_class_key __key; \
        __INIT_WORK_KEY(_work, _func, _onstack, &__key); \
    } while (0)

#define INIT_WORK(_work, _func) \
    __INIT_WORK((_work), (_func), 0)

int queue_work(struct workqueue_struct *wq, struct work_struct *work);
int queue_work_on(int cpu, struct workqueue_struct *wq, struct work_struct *work);
int queue_delayed_work(struct workqueue_struct *wq, struct work_struct *work, unsigned long delay);

```

Листинг 28 — Инициализация очереди работ

Выделение и освобождение номеров устройств

```

extern int alloc_chrdev_region(dev_t *, unsigned, unsigned, const char *);
extern int register_chrdev_region(dev_t, unsigned, const char *);
extern int __register_chrdev(unsigned int major, unsigned int baseminor,
    unsigned int count, const char *name,
    const struct file_operations *fops);
extern void __unregister_chrdev(unsigned int major, unsigned int baseminor,
    unsigned int count, const char *name);
extern void unregister_chrdev_region(dev_t, unsigned);
extern void chrdev_show(struct seq_file *, off_t);

static inline int register_chrdev(unsigned int major, const char *name,
    const struct file_operations *fops)
{
    return __register_chrdev(major, 0, 256, name, fops);
}

static inline void unregister_chrdev(unsigned int major, const char *name)
{
    __unregister_chrdev(major, 0, 256, name);
}

MAJOR(dev_t dev); /* get major number */
MINOR(dev_t dev); /* get minor number */

```

```
/* Having major and minor numbers, they can be combined into dev_t: */
MKDEV(int major, int minor); /* combine into dev_t */
```

Листинг 29 — Выделение и освобождение номеров устройств

printk flags

```
#define KERN_EMERG KERN_SOH "0" /* system is unusable */
#define KERN_ALERT KERN_SOH "1" /* action must be taken immediately */
#define KERN_CRIT KERN_SOH "2" /* critical conditions */
#define KERN_ERR KERN_SOH "3" /* error conditions */
#define KERN_WARNING KERN_SOH "4" /* warning conditions */
#define KERN_NOTICE KERN_SOH "5" /* normal but significant condition */
#define KERN_INFO KERN_SOH "6" /* informational */
#define KERN_DEBUG KERN_SOH "7" /* debug-level messages */
}
```

Листинг 30 — printk flags

task_struct

```
struct task_struct {
#ifdef CONFIG_THREAD_INFO_IN_TASK
    struct thread_info thread_info;
#endif
    unsigned int __state;
    randomized_struct_fields_start

    void *stack;
    refcount_t usage;
    /* Per task flags (PF_*), defined further below: */
    unsigned int flags;
    unsigned int ptrace;

#ifdef CONFIG_MEM_ALLOC_PROFILING
    struct alloc_tag *alloc_tag;
#endif

    int on_cpu;
    struct __call_single_node wake_entry;
    unsigned int wakee_flips;
    unsigned long wakee_flip_decay_ts;
    struct task_struct *last_wakee;

    int recent_used_cpu;
    int wake_cpu;
    int on_rq;

    int prio;
    int static_prio;
    int normal_prio;
    ...
} __attribute__((aligned (64)));
```

Листинг 31 — task_struct

Загрузка модуля

```
#include <linux/module.h>
#include <linux/init.h>

static int __init md_init(void)
{
    printk("Module loaded!\n");
    return 0;
}

static int __init md_init(void)
{
    struct task_struct *task = &init_task;
    do {
        printk(KERN_INFO "Task: pid: %d, comm: %s, ppid: %d, state: %ld,
flags: %x, prio: %d, policy: %d, root: %s, pwd: %s\n",
            task->pid, task->comm, task->parent->pid,
            task->state, task->flags, task->prio, task->policy,
            task->fs->root.dentry->d_iname, task->fs->pwd.dentry-
>d_name.name);
    } while ((task = next_task(task)) != &init_task);
    printk(KERN_INFO "Current: pid: %d, comm: %s, ppid: %d, state: %ld,
flags: %x, prio: %d, policy: %d, root: %s, pwd: %s\n",
        current->pid, current->comm, current->parent->pid,
        current->state, current->flags, current->prio, current->policy,
        current->fs->root.dentry->d_iname, current->fs->pwd.dentry-
>d_name.name);
    return 0;
}

static void __exit md_exit(void)
{
    printk("Module unloaded\n");
}

module_init(md_init);
module_exit(md_exit);
}
```

Листинг 32 — Загрузка модуля

Мультиплексор

```
#define _GNU_SOURCE
#include <errno.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/epoll.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <sys/socket.h>
#include <sys/stat.h>
#include <time.h>
```

```

    fmode_t          f_mode;          /* Access mode. */
    atomic_long_t    f_count;         /* File reference counter. */
    loff_t           f_pos;           /* Logical position in the file:
file is assumed to start at offset 0; each read/write operation changes
this field by the corresponding number of bytes. */

    ...
    struct path      f_path;          /* Link to dentry and mounted file
system. */
    struct inode      *f_inode;       /* Pointer to the inode (physical
file). */
    const struct file_operations *f_op; /* Operations defined on an open
file. */
    ...
};

```

Листинг 15 — struct file

struct file_operations

```

struct file_operations {
    struct module *owner;
    fop_flags_t fop_flags;
    loff_t (*llseek) (struct file *, loff_t, int);
    ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);
    ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);
    ssize_t (*read_iter) (struct kiocb *, struct iov_iter *);
    ssize_t (*write_iter) (struct kiocb *, struct iov_iter *);
    int (*iopoll)(struct kiocb *kiocb, struct io_comp_batch *,
        unsigned int flags);
    int (*iterate_shared) (struct file *, struct dir_context *);
    __poll_t (*poll) (struct file *, struct poll_table_struct *);
    long (*unlocked_ioctl) (struct file *, unsigned int, unsigned long);
    long (*compat_ioctl) (struct file *, unsigned int, unsigned long);
    int (*mmap) (struct file *, struct vm_area_struct *);
    int (*open) (struct inode *, struct file *);
    int (*flush) (struct file *, fl_owner_t id);
    int (*release) (struct inode *, struct file *);
    int (*fsync) (struct file *, loff_t, loff_t, int datasync);
    int (*fasync) (int, struct file *, int);
    int (*lock) (struct file *, int, struct file_lock *);
    unsigned long (*get_unmapped_area)(struct file *, unsigned long,
unsigned long, unsigned long, unsigned long);
    int (*check_flags)(int);
    int (*flock) (struct file *, int, struct file_lock *);
    ssize_t (*splice_write)(struct pipe_inode_info *, struct file *, loff_t
*, size_t, unsigned int);
    ssize_t (*splice_read)(struct file *, loff_t *, struct pipe_inode_info
*, size_t, unsigned int);
    void (*splice_eof)(struct file *file);
    int (*setlease)(struct file *, int, struct file_lease **, void **);
    long (*fallocate)(struct file *file, int mode, loff_t offset,
        loff_t len);
    void (*show_fdinfo)(struct seq_file *m, struct file *f);
#ifdef CONFIG_MMU
    unsigned (*mmap_capabilities)(struct file *);
#endif
};

```



```

ssize_t (*copy_file_range)(struct file *, loff_t, struct file *,
    loff_t, size_t, unsigned int);
loff_t (*remap_file_range)(struct file *file_in, loff_t pos_in,
    struct file *file_out, loff_t pos_out,
    loff_t len, unsigned int remap_flags);
int (*fadvise)(struct file *, loff_t, loff_t, int);
int (*uring_cmd)(struct io_uring_cmd *ioucmd, unsigned int issue_flags);
int (*uring_cmd_iopoll)(struct io_uring_cmd *, struct io_comp_batch *,
    unsigned int poll_flags);
int (*mmap_prepare)(struct vm_area_desc *);
}

```

Листинг 16 — struct file_operations

open

```

int open(const char *pathname, int flags); /* pathname — Name of the file
to open. flags — Flags defining access mode and behavior. */
int open(const char *pathname, int flags, mode_t mode); /* mode — Access
permissions for the created file. The third parameter is required only
when open() is called to create a new file (with O_CREAT). If the file
already exists, mode is ignored. */

/* Returns a file descriptor — a non-negative integer, which is the lowest
 * unused descriptor in the process's open file table. Upon creation of a
new file, a new inode is initialized. When a file is opened, the struct
file fields (including f_pos) are initialized, with the file offset set to
the beginning of the file.
 */

/*
 * Access flags (one of them must be specified):
 *   O_RDONLY — Open file for reading only.
 *   O_WRONLY — Open file for writing only.
 *   O_RDWR  — Open file for reading and writing.
 *
 * Creation and behavior flags:
 *   O_CREAT   — Create the file if it does not exist. When this flag
is used, the mode parameter must be provided to set access permissions.
 *   O_APPEND  — Append to the existing file: before each write() call,
the file offset is moved to the end, so previously written data is
not overwritten.
 *   O_TRUNC   — If the file exists and is opened for writing, truncate
it to zero length.
 *   O_NONBLOCK — Set non-blocking mode. Useful for special files like
FIFOs and terminals; open() returns immediately instead of blocking.
Also affects read/write operations, allowing them to return
immediately with an error if no input is available or output cannot
be written.
 *   O_CLOEXEC  — Set the close-on-exec flag on the new file descriptor.
 *   O_PATH     — Obtain only a file descriptor (the file itself is not
actually opened for reading/writing).
 *   O_TMPFILE  — Create an unnamed temporary regular file.
 *
 * MODE (permissions for O_CREAT):
 * When creating a new file, the owner and group IDs are taken from the

```



```

* process's effective IDs. The following symbolic constants are
available:
*   S_IRWXU / S_IRWXG / S_IRWXO – Read, write, and execute permissions
    for owner / group / others.
*   S_IRUSR / S_IRGRP / S_IROTH – Read permission.
*   S_IWUSR / S_IWGRP / S_IWOTH – Write permission.
*   S_IXUSR / S_IXGRP / S_IXOTH – Execute permission.
*/

```

Листинг 17 – open

struct FILE

```

typedef struct _IO_FILE FILE;

FILE *fopen(const char *fname, const char *mode);

```

Листинг 18 – struct FILE

struct _IO_FILE

```

struct _IO_FILE {
    int _flags; /* High-order word is _IO_MAGIC; rest is flags. */
#define _IO_file_flags _flags

    /* The following pointers correspond to the C++ streambuf protocol. */
    /* Note: Tk uses the _IO_read_ptr and _IO_read_end fields directly. */
    char* _IO_read_ptr; /* Current read pointer */
    char* _IO_read_end; /* End of get area. */
    char* _IO_read_base; /* Start of putback+get area. */
    char* _IO_write_base; /* Start of put area. */
    char* _IO_write_ptr; /* Current put pointer. */
    char* _IO_write_end; /* End of put area. */
    char* _IO_buf_base; /* Start of reserve area. */
    char* _IO_buf_end; /* End of reserve area. */
    /* The following fields are used to support backing up and undo. */
    char *_IO_save_base; /* Pointer to start of non-current get area. */
    char *_IO_backup_base; /* Pointer to first valid character of backup
area */
    char *_IO_save_end; /* Pointer to end of non-current get area. */

    struct _IO_marker *_markers;
    struct _IO_FILE *_chain;
    int _fileno;
#if 0
    int _blksize;
#else
    int _flags2;
#endif
    _IO_off_t _old_offset; /* This used to be _offset but it's too small.
*/

#define __HAVE_COLUMN /* temporary */
    /* 1+column number of pbase(); 0 is unknown. */
    unsigned short _cur_column;
    signed char _vtable_offset;

```



```

char _shortbuf[1];

/* char* _save_gptr; char* _save_egptr; */

_IO_lock_t *_lock;
#ifdef _IO_USE_OLD_IO_FILE
};

```

Листинг 19 — struct IO_FILE

struct stat

```

struct stat {
    dev_t      st_dev;          /* [XSI] ID of device containing file */
    ino_t      st_ino;          /* [XSI] File serial number */
    mode_t     st_mode;         /* [XSI] Mode of file (see below) */
    nlink_t    st_nlink;        /* [XSI] Number of hard links */
    uid_t      st_uid;          /* [XSI] User ID of the file */
    gid_t      st_gid;          /* [XSI] Group ID of the file */
    dev_t      st_rdev;         /* [XSI] Device ID */
#ifdef _POSIX_C_SOURCE || defined(_DARWIN_C_SOURCE)
    struct timespec st_atimespec; /* time of last access */
    struct timespec st_mtimespec; /* time of last data modification */
    struct timespec st_ctimespec; /* time of last status change */
#else
    time_t      st_atime;        /* [XSI] Time of last access */
    long        st_atimensec;    /* nsec of last access */
    time_t      st_mtime;        /* [XSI] Last data modification time */
    long        st_mtimensec;    /* last data modification nsec */
    time_t      st_ctime;        /* [XSI] Time of last status change */
    long        st_ctimensec;    /* nsec of last status change */
#endif
    off_t      st_size;         /* [XSI] file size, in bytes */
    blkcnt_t   st_blocks;       /* [XSI] blocks allocated for file */
    blksize_t  st_blksize;      /* [XSI] optimal blocksize for I/O */
    __uint32_t st_flags;         /* user defined flags for file */
    __uint32_t st_gen;           /* file generation number */
    __int32_t  st_lspare;        /* RESERVED: DO NOT USE! */
    __int64_t  st_qspare[2];     /* RESERVED: DO NOT USE! */
};

```

Листинг 20 — struct stat

Обработчики аппаратных прерываний

```

/**
 * enum irqreturn - irqreturn type values
 * @IRQ_NONE:   interrupt was not from this device or was not handled
 * @IRQ_HANDLED: interrupt was handled by this device
 * @IRQ_WAKE_THREAD: handler requests to wake the handler thread
 */
enum irqreturn {
    IRQ_NONE      = (0 << 0),
    IRQ_HANDLED   = (1 << 0),
    IRQ_WAKE_THREAD = (1 << 1),
};

```



```
typedef enum irqreturn_t;
}
```

Листинг 21 — Обработчики аппаратных прерываний

Флаги IRQ

```
/*
 * IRQF_SHARED - allow sharing the irq among several devices
 * IRQF_PROBE_SHARED - set by callers when they expect sharing mismatches
to occur
 * IRQF_TIMER - Flag to mark this interrupt as timer interrupt
 * IRQF_PERCPU - Interrupt is per cpu
 * IRQF_NOBALANCING - Flag to exclude this interrupt from irq balancing
 * IRQF_IRQPOLL - Interrupt is used for polling (only the interrupt that
is
 *
    registered first in a shared interrupt is considered for
 *
    performance reasons)
 * IRQF_ONESHOT - Interrupt is not reenabled after the hardirq handler
finished.
 *
    Used by threaded interrupts which need to keep the
 *
    irq line disabled until the threaded handler has been
run.
 * IRQF_NO_SUSPEND - Do not disable this IRQ during suspend. Does not
guarantee
 *
    that this interrupt will wake the system from a
suspended
 *
    state. See Documentation/power/suspend-and-
interrupts.rst
 * IRQF_FORCE_RESUME - Force enable it on resume even if IRQF_NO_SUSPEND
is set
 * IRQF_NO_THREAD - Interrupt cannot be threaded
 * IRQF_EARLY_RESUME - Resume IRQ early during syscore instead of at
device
 *
    resume time.
 * IRQF_COND_SUSPEND - If the IRQ is shared with a NO_SUSPEND user,
execute this
 *
    interrupt handler after suspending interrupts. For
system
 *
    wakeup devices users need to implement wakeup detection
in
 *
    their interrupt handlers.
 * IRQF_NO_AUTOEN - Don't enable IRQ or NMI automatically when users
request it.
 *
    Users will enable it explicitly by enable_irq() or
enable_nmi()
 *
    later.
 * IRQF_NO_DEBUG - Exclude from runaway detection for IPI and similar
handlers,
 *
    depends on IRQF_PERCPU.
 * IRQF_COND_ONESHOT - Agree to do IRQF_ONESHOT if already set for a
shared
 *
    interrupt.
 */
#define IRQF_SHARED 0x00000080
```



```

#define IRQF_PROBE_SHARED 0x00000100
#define __IRQF_TIMER      0x00000200
#define IRQF_PERCPU       0x00000400
#define IRQF_NOBALANCING  0x00000800
#define IRQF_IRQPOLL      0x00001000
#define IRQF_ONESHOT      0x00002000
#define IRQF_NO_SUSPEND   0x00004000
#define IRQF_FORCE_RESUME 0x00008000
#define IRQF_NO_THREAD     0x00010000
#define IRQF_EARLY_RESUME 0x00020000
#define IRQF_COND_SUSPEND 0x00040000
#define IRQF_NO_AUTOEN    0x00080000
#define IRQF_NO_DEBUG     0x00100000
#define IRQF_COND_ONESHOT 0x00200000

```

Листинг 22 — Флаги IRQ

struct irqaction

```

/**
 * struct irqaction - per interrupt action descriptor
 * @handler: interrupt handler function
 * @name: name of the device
 * @dev_id: cookie to identify the device
 * @percpu_dev_id: cookie to identify the device
 * @affinity: CPUs this irqaction is allowed to run on
 * @next: pointer to the next irqaction for shared interrupts
 * @irq: interrupt number
 * @flags: flags (see IRQF_* above)
 * @thread_fn: interrupt handler function for threaded interrupts
 * @thread: thread pointer for threaded interrupts
 * @secondary: pointer to secondary irqaction (force threading)
 * @thread_flags: flags related to @thread
 * @thread_mask: bitmask for keeping track of @thread activity
 * @dir: pointer to the proc/irq/NN/name entry
 */
struct irqaction {
    irq_handler_t handler;
    union {
        void *dev_id;
        void __percpu *percpu_dev_id;
    };
    const struct cpumask *affinity;
    struct irqaction *next;
    irq_handler_t thread_fn;
    struct task_struct *thread;
    struct irqaction *secondary;
    unsigned int irq;
    unsigned int flags;
    unsigned long thread_flags;
    unsigned long thread_mask;
    const char *name;
    struct proc_dir_entry *dir;
}

/**

```



```

* request_irq - Add a handler for an interrupt line
* @irq: The interrupt line to allocate
* @handler: Function to be called when the IRQ occurs.
*   Primary handler for threaded interrupts
*   If NULL, the default primary handler is installed
* @flags: Handling flags
* @name: Name of the device generating this interrupt
* @dev: A cookie passed to the handler function
*
* This call allocates an interrupt and establishes a handler; see
* the documentation for request_threaded_irq() for details.
*/

static inline int __must_check
request_irq(unsigned int irq, irq_handler_t handler, unsigned long flags,
            const char *name, void *dev)
{
    return request_threaded_irq(irq, handler, NULL, flags |
IRQF_COND_ONESHOT, name, dev);
}

extern const void *free_irq(unsigned int, void *); /* free irq line from
handler */

```

Листинг 23 — struct irqaction

softirq

```

/* PLEASE, avoid to allocate new softirqs, if you need not _really_ high
frequency threaded job scheduling. For almost all the purposes
tasklets are more than enough. F.e. all serial device BHs et
al. should be converted to tasklets, not to softirqs.
*/

enum
{
    HI_SOFTIRQ=0,
    TIMER_SOFTIRQ,
    NET_TX_SOFTIRQ, /* network */
    NET_RX_SOFTIRQ, /* network */
    BLOCK_SOFTIRQ,
    IRQ_POLL_SOFTIRQ,
    TASKLET_SOFTIRQ,
    SCHED_SOFTIRQ,
    HRTIMER_SOFTIRQ,
    RCU_SOFTIRQ, /* Preferable RCU should always be the last softirq */

    NR_SOFTIRQS
};

struct softirq_action {
    void (*action)(struct softirq_action *); /* executable function */
};

static struct softirq_action softirq_vec[NR_SOFTIRQS];

```

62 / 289

63 / 289

64 / 289

65 / 289

66 / 289

67 / 289

69 / 289

70 / 289

71 / 289

```

const char * const softirq_to_name[NR_SOFTIRQS] = {
    "HI", "TIMER", "NET_TX", "NET_RX", "BLOCK", "IRQ_POLL", "TASKLET",
    "SCHED", "HRTIMER", "RCU"
};

void raise_softirq(unsigned int nr); /* Marks softirq as pending on
current CPU (called from HW interrupt handler) */

void open_softirq(int nr, void (*action)(void)); /* Registers softirq
handler (called during initialization) */

/*
 * open_softirq - registers handler (nr = index in softirq_vec, action =
handler function)
 * raise_softirq - marks registered softirq as pending so it can be
executed
 */

```

Листинг 24 — softirq

tasklet

```

/* Tasklets --- multithreaded analogue of BHs.
Main feature differing them of generic softirqs: tasklet
is running only on one CPU simultaneously.

Main feature differing them of BHs: different tasklets
may be run simultaneously on different CPUs.

Properties:
 * If tasklet_schedule() is called, then tasklet is guaranteed
to be executed on some cpu at least once after this.
 * If the tasklet is already scheduled, but its execution is still not
started, it will be executed only once.
 * If this tasklet is already running on another CPU (or schedule is
called
from tasklet itself), it is rescheduled for later.
 * Tasklet is strictly serialized wrt itself, but not
wrt another tasklets. If client needs some intertask synchronization,
he makes it with spinlocks.
 */

struct tasklet_struct
{
    struct tasklet_struct *next;
    unsigned long state;
    atomic_t count;
    bool use_callback;
    union {
        void (*func)(unsigned long data);
        void (*callback)(struct tasklet_struct *t);
    };
    unsigned long data;
};

```



```

enum
{
    TASKLET_STATE_SCHED, /* Tasklet is scheduled for execution */
    TASKLET_STATE_RUN /* Tasklet is running (SMP only) */
};

#define DECLARE_TASKLET(name, _callback) \
struct tasklet_struct name = { \
    .count = ATOMIC_INIT(0), \
    .callback = _callback, \
    .use_callback = true, \
}

#define DECLARE_TASKLET_DISABLED(name, _callback) \
struct tasklet_struct name = { \
    .count = ATOMIC_INIT(1), \
    .callback = _callback, \
    .use_callback = true, \
}

#define from_tasklet(var, callback_tasklet, tasklet_fieldname) \
    container_of(callback_tasklet, typeof(*var), tasklet_fieldname)

void tasklet_init(struct tasklet_struct *t,
    void (*func)(unsigned long), unsigned long data)
{
    t->next = NULL;
    t->state = 0;
    atomic_set(&t->count, 0);
    t->func = func;
    t->use_callback = false;
    t->data = data;
}

static void __tasklet_schedule_common(struct tasklet_struct *t,
    struct tasklet_head __percpu *headp,
    unsigned int softirq_nr)
{
    struct tasklet_head *head;
    unsigned long flags;

    local_irq_save(flags);
    head = this_cpu_ptr(headp);
    t->next = NULL;
    *head->tail = t;
    head->tail = &(t->next);
    raise_softirq_irqoff(softirq_nr);
    local_irq_restore(flags);
}

void __tasklet_schedule(struct tasklet_struct *t)
{
    __tasklet_schedule_common(t, &tasklet_vec,
        TASKLET_SOFTIRQ);
}

```



```
EXPORT_SYMBOL(__tasklet_schedule);

void __tasklet_hi_schedule(struct tasklet_struct *t)
{
    __tasklet_schedule_common(t, &tasklet_hi_vec,
                              HI_SOFTIRQ);
}
EXPORT_SYMBOL(__tasklet_hi_schedule);
```

Листинг 25 — tasklet

struct worker

```
struct worker_pool;
/*
 * The poor guys doing the actual heavy lifting. All on-duty workers are
 * either serving the manager role, on idle list or on busy hash. For
 * details on the locking annotation (L, I, X...), refer to workqueue.c.
 * Only to be used in workqueue and async.
 */
struct worker {
    /* on idle list while idle, on busy hash table while busy */
    union {
        struct list_head entry; /* L: while idle */
        struct hlist_node hentry; /* L: while busy */
    };

    struct work_struct *current_work; /* K: work being processed and its
 */
    work_func_t current_func; /* K: function */
    struct pool_workqueue *current_pwq; /* K: pwq */
    u64 current_at; /* K: runtime at start or last wakeup */
    unsigned long current_start; /* K: start time of current work item */
    unsigned int current_color; /* K: color */

    int sleeping; /* S: is worker sleeping? */

    /* used by the scheduler to determine a worker's last known identity */
    work_func_t last_func; /* K: last work's fn */

    struct list_head scheduled; /* L: scheduled works */

    struct task_struct *task; /* I: worker task */
    struct worker_pool *pool; /* A: the associated pool */
    /* L: for rescuers */
    struct list_head node; /* A: anchored at pool->workers */
    /* A: runs through worker->node */

    unsigned long last_active; /* K: last active timestamp */
    unsigned int flags; /* L: flags */
    int id; /* I: worker id */

    /*
     * Opaque string set with work_set_desc(). Printed out with task
     * dump for debugging - WARN, BUG, panic or sysrq.
     */
};
```

21	...
22	...
23	...
24	...
25	...
26	...
27	...
28	...
29	...
30	...
31	...
32	...
33	...
34	...
35	...
36	...
37	...
38	...
39	...
40	...
41	...
42	...
43	...
44	...
45	...
46	...
47	...
48	...
49	...
50	...
51	...
52	...
53	...
54	...
55	...
56	...
57	...
58	...
59	...
60	...
61	...
62	...
63	...
64	...
65	...
66	...
67	...
68	...
69	...
70	...
71	...
72	...
73	...
74	...
75	...
76	...
77	...
78	...
79	...
80	...
81	...
82	...
83	...
84	...
85	...
86	...
87	...
88	...
89	...
90	...
91	...
92	...
93	...
94	...
95	...
96	...
97	...
98	...
99	...
100	...

92 / 289

6.4-файловая подсистема LINUX: поддержка большого числа файловых систем и структура, описывающая файловую систему

[illegible]

```

1 struct file_system_type {
2     const char *name; // generic name (FS, HIDEFS, DRV, ...).
3     int fs_flags;
4     struct dentry *(*mount) (struct file_system_type * fs, const char *, void *);
5     void (*kill_sb) (struct super_block *);
6     struct module *owner;
7     struct file_system_type *next;
8     struct hlist_head *sgetops;
9     ...
10 };

```

[illegible]

6.5 УЗС: четыре основные структуры файловой системы и связь между ними

В основе архитектуры УЗС лежат 4 структуры (по другим структурам):

- 1) struct super_block;
- 2) struct inode;
- 3) struct dentry;
- 4) struct file.

The diagram illustrates the Chrysopyga Virtual File System architecture. It is divided into three main layers: user space, filesystem, and physical layer. In the user space, there is a 'win32' component that interacts with a 'vfs' (Virtual File System) component. The 'vfs' component calls the 'filesystem.write' method in the filesystem layer. The filesystem layer is represented by a cylinder icon. The physical layer contains a table of file types: ext, yfs, fat, dos, nfs, dev, bin, etc, and proc. A 'chrysopyga' component is shown in the user space, interacting with the 'vfs' and 'filesystem' layers. A 'chrysopyga vfs' component is also shown in the user space, interacting with the 'vfs' and 'filesystem' layers. A 'chrysopyga' component is also shown in the physical layer, interacting with the 'filesystem' and 'physical layer'.

[illegible]

Символьный
↓
Базовый
↓
Проверка прав доступа
↓
Логический
↓
Физический

Символьный уровень – типизация файлов и файловых систем по типу файловой системы (FAT, FAT32, NTFS, HFS, UFS, ReiserFS, Ext2, Ext3, XFS, JFS, ZFS, Btrfs, F2FS, APFS, HFS+, FFS, UFS2, UFS2+, UFS3, UFS3+, UFS4, UFS4+, UFS5, UFS5+, UFS6, UFS6+, UFS7, UFS7+, UFS8, UFS8+, UFS9, UFS9+, UFS10, UFS10+, UFS11, UFS11+, UFS12, UFS12+, UFS13, UFS13+, UFS14, UFS14+, UFS15, UFS15+, UFS16, UFS16+, UFS17, UFS17+, UFS18, UFS18+, UFS19, UFS19+, UFS20, UFS20+, UFS21, UFS21+, UFS22, UFS22+, UFS23, UFS23+, UFS24, UFS24+, UFS25, UFS25+, UFS26, UFS26+, UFS27, UFS27+, UFS28, UFS28+, UFS29, UFS29+, UFS30, UFS30+, UFS31, UFS31+, UFS32, UFS32+, UFS33, UFS33+, UFS34, UFS34+, UFS35, UFS35+, UFS36, UFS36+, UFS37, UFS37+, UFS38, UFS38+, UFS39, UFS39+, UFS40, UFS40+, UFS41, UFS41+, UFS42, UFS42+, UFS43, UFS43+, UFS44, UFS44+, UFS45, UFS45+, UFS46, UFS46+, UFS47, UFS47+, UFS48, UFS48+, UFS49, UFS49+, UFS50, UFS50+, UFS51, UFS51+, UFS52, UFS52+, UFS53, UFS53+, UFS54, UFS54+, UFS55, UFS55+, UFS56, UFS56+, UFS57, UFS57+, UFS58, UFS58+, UFS59, UFS59+, UFS60, UFS60+, UFS61, UFS61+, UFS62, UFS62+, UFS63, UFS63+, UFS64, UFS64+, UFS65, UFS65+, UFS66, UFS66+, UFS67, UFS67+, UFS68, UFS68+, UFS69, UFS69+, UFS70, UFS70+, UFS71, UFS71+, UFS72, UFS72+, UFS73, UFS73+, UFS74, UFS74+, UFS75, UFS75+, UFS76, UFS76+, UFS77, UFS77+, UFS78, UFS78+, UFS79, UFS79+, UFS80, UFS80+, UFS81, UFS81+, UFS82, UFS82+, UFS83, UFS83+, UFS84, UFS84+, UFS85, UFS85+, UFS86, UFS86+, UFS87, UFS87+, UFS88, UFS88+, UFS89, UFS89+, UFS90, UFS90+, UFS91, UFS91+, UFS92, UFS92+, UFS93, UFS93+, UFS94, UFS94+, UFS95, UFS95+, UFS96, UFS96+, UFS97, UFS97+, UFS98, UFS98+, UFS99, UFS99+, UFS100, UFS100+, UFS101, UFS101+, UFS102, UFS102+, UFS103, UFS103+, UFS104, UFS104+, UFS105, UFS105+, UFS106, UFS106+, UFS107, UFS107+, UFS108, UFS108+, UFS109, UFS109+, UFS110, UFS110+, UFS111, UFS111+, UFS112, UFS112+, UFS113, UFS113+, UFS114, UFS114+, UFS115, UFS115+, UFS116, UFS116+, UFS117, UFS117+, UFS118, UFS118+, UFS119, UFS119+, UFS120, UFS120+, UFS121, UFS121+, UFS122, UFS122+, UFS123, UFS123+, UFS124, UFS124+, UFS125, UFS125+, UFS126, UFS126+, UFS127, UFS127+, UFS128, UFS128+, UFS129, UFS129+, UFS130, UFS130+, UFS131, UFS131+, UFS132, UFS132+, UFS133, UFS133+, UFS134, UFS134+, UFS135, UFS135+, UFS136, UFS136+, UFS137, UFS137+, UFS138, UFS138+, UFS139, UFS139+, UFS140, UFS140+, UFS141, UFS141+, UFS142, UFS142+, UFS143, UFS143+, UFS144, UFS144+, UFS145, UFS145+, UFS146, UFS146+, UFS147, UFS147+, UFS148, UFS148+, UFS149, UFS149+, UFS150, UFS150+, UFS151, UFS151+, UFS152, UFS152+, UFS153, UFS153+, UFS154, UFS154+, UFS155, UFS155+, UFS156, UFS156+, UFS157, UFS157+, UFS158, UFS158+, UFS159, UFS159+, UFS160, UFS160+, UFS161, UFS161+, UFS162, UFS162+, UFS163, UFS163+, UFS164, UFS164+, UFS165, UFS165+, UFS166, UFS166+, UFS167, UFS167+, UFS168, UFS168+, UFS169, UFS169+, UFS170, UFS170+, UFS171, UFS171+, UFS172, UFS172+, UFS173, UFS173+, UFS174, UFS174+, UFS175, UFS175+, UFS176, UFS176+, UFS177, UFS177+, UFS178, UFS178+, UFS179, UFS179+, UFS180, UFS180+, UFS181, UFS181+, UFS182, UFS182+, UFS183, UFS183+, UFS184, UFS184+, UFS185, UFS185+, UFS186, UFS186+, UFS187, UFS187+, UFS188, UFS188+, UFS189, UFS189+, UFS190, UFS190+, UFS191, UFS191+, UFS192, UFS192+, UFS193, UFS193+, UFS194, UFS194+, UFS195, UFS195+, UFS196, UFS196+, UFS197, UFS197+, UFS198, UFS198+, UFS199, UFS199+, UFS200, UFS200+, UFS201, UFS201+, UFS202, UFS202+, UFS203, UFS203+, UFS204, UFS204+, UFS205, UFS205+, UFS206, UFS206+, UFS207, UFS207+, UFS208, UFS208+, UFS209, UFS209+, UFS210, UFS210+, UFS211, UFS211+, UFS212, UFS212+, UFS213, UFS213+, UFS214, UFS214+, UFS215, UFS215+, UFS216, UFS216+, UFS217, UFS217+, UFS218, UFS218+, UFS219, UFS219+, UFS220, UFS220+, UFS221, UFS221+, UFS222, UFS222+, UFS223, UFS223+, UFS224, UFS224+, UFS225, UFS225+, UFS226, UFS226+, UFS227, UFS227+, UFS228, UFS228+, UFS229, UFS229+, UFS230, UFS230+, UFS231, UFS231+, UFS232, UFS232+, UFS233, UFS233+, UFS234, UFS234+, UFS235, UFS235+, UFS236, UFS236+, UFS237, UFS237+, UFS238, UFS238+, UFS239, UFS239+, UFS240, UFS240+, UFS241, UFS241+, UFS242, UFS242+, UFS243, UFS243+, UFS244, UFS244+, UFS245, UFS245+, UFS246, UFS246+, UFS247, UFS247+, UFS248, UFS248+, UFS249, UFS249+, UFS250, UFS250+, UFS251, UFS251+, UFS252, UFS252+, UFS253, UFS253+, UFS254, UFS254+, UFS255, UFS255+, UFS256, UFS256+, UFS257, UFS257+, UFS258, UFS258+, UFS259, UFS259+, UFS260, UFS260+, UFS261, UFS261+, UFS262, UFS262+, UFS263, UFS263+, UFS264, UFS264+, UFS265, UFS265+, UFS266, UFS266+, UFS267, UFS267+, UFS268, UFS268+, UFS269, UFS269+, UFS270, UFS270+, UFS271, UFS271+, UFS272, UFS272+, UFS273, UFS273+, UFS274, UFS274+, UFS275, UFS275+, UFS276, UFS276+, UFS277, UFS277+, UFS278, UFS278+, UFS279, UFS279+, UFS280, UFS280+, UFS281, UFS281+, UFS282, UFS282+, UFS283, UFS283+, UFS284, UFS284+, UFS285, UFS285+, UFS286, UFS286+, UFS287, UFS287+, UFS288, UFS288+, UFS289, UFS289+, UFS290, UFS290+, UFS291, UFS291+, UFS292, UFS292+, UFS293, UFS293+, UFS294, UFS294+, UFS295, UFS295+, UFS296, UFS296+, UFS297, UFS297+, UFS298, UFS298+, UFS299, UFS299+, UFS300, UFS300+, UFS301, UFS301+, UFS302, UFS302+, UFS303, UFS303+, UFS304, UFS304+, UFS305, UFS305+, UFS306, UFS306+, UFS307, UFS307+, UFS308, UFS308+, UFS309, UFS309+, UFS310, UFS310+, UFS311, UFS311+, UFS312, UFS312+, UFS313, UFS313+, UFS314, UFS314+, UFS315, UFS315+, UFS316, UFS316+, UFS317, UFS317+, UFS318, UFS318+, UFS319, UFS319+, UFS320, UFS320+, UFS321, UFS321+, UFS322, UFS322+, UFS323, UFS323+, UFS324, UFS324+, UFS325, UFS325+, UFS326, UFS326+, UFS327, UFS327+, UFS328, UFS328+, UFS329, UFS329+, UFS330, UFS330+, UFS331, UFS331+, UFS332, UFS332+, UFS333, UFS333+, UFS334, UFS334+, UFS335, UFS335+, UFS336, UFS336+, UFS337, UFS337+, UFS338, UFS338+, UFS339, UFS339+, UFS340, UFS340+, UFS341, UFS341+, UFS342, UFS342+, UFS343, UFS343+, UFS344, UFS344+, UFS345, UFS345+, UFS346, U

1	/
2	bin
3	etc
4	dev
5	src
6	local
7	lib

[illegible]

101 / 289

```

char        desc[WORKER_DESC_LEN];

/* used only by rescuers to point to the target workqueue */
struct workqueue_struct *rescue_wq; /* I: the workqueue to rescue */
};

```

Листинг 26 — struct worker

struct worker_pool

```

struct worker_pool {
    raw_spinlock_t    lock;    /* the pool lock */
    int               cpu;    /* I: the associated cpu */
    int               node;    /* I: the associated node ID */
    int               id;    /* I: pool ID */
    unsigned int      flags;    /* L: flags */

    unsigned long     last_progress_ts; /* L: last forward progress timestamp */
    bool              cpu_stall; /* WD: stalled cpu bound pool */
    /*
     * The counter is incremented in a process context on the associated CPU
     * w/ preemption disabled, and decremented or reset in the same context
     * but w/ pool->lock held. The readers grab pool->lock and are
     * guaranteed to see if the counter reached zero.
     */
    int               nr_running;

    struct list_head   worklist; /* L: list of pending works */

    int               nr_workers; /* L: total number of workers */
    int               nr_idle; /* L: currently idle workers */

    struct list_head   idle_list; /* L: list of idle workers */
    struct timer_list  idle_timer; /* L: worker idle timeout */
    struct work_struct  idle_cull_work; /* L: worker idle cleanup */

    struct timer_list  mayday_timer; /* L: SOS timer for workers */

    /* a workers is either on busy_hash or idle_list, or the manager */
    DECLARE_HASHTABLE(busy_hash, BUSY_WORKER_HASH_ORDER);
    /* L: hash of busy workers */

    struct worker      *manager; /* L: purely informational */
    struct list_head    workers; /* A: attached workers */

    struct ida          worker_ida; /* worker IDs for task name */

    struct workqueue_attrs *attrs; /* I: worker attributes */
    struct hlist_node   hash_node; /* PL: unbound_pool_hash node */
    int                 refcnt; /* PL: refcnt for unbound pools */
#ifdef CONFIG_PREEMPT_RT
    spinlock_t          cb_lock; /* BH worker cancel lock */
#endif
    /*
     * Destruction of pool is RCU protected to allow dereferences
     */
};

```



```

    * from get_work_pool().
    */
    struct rcu_head    rcu;
};

```

Листинг 27 — struct worker_pool

Инициализация очереди работ

```

#define DECLARE_WORK(n, f) \
    struct work_struct n = __WORK_INITIALIZER(n, f)

#define __INIT_WORK(_work, _func, _onstack) \
    do { \
        static __maybe_unused struct lock_class_key __key; \
        __INIT_WORK_KEY(_work, _func, _onstack, &__key); \
    } while (0)

#define INIT_WORK(_work, _func) \
    __INIT_WORK((_work), (_func), 0)

int queue_work(struct workqueue_struct *wq, struct work_struct *work);
int queue_work_on(int cpu, struct workqueue_struct *wq, struct work_struct *work);
int queue_delayed_work(struct workqueue_struct *wq, struct work_struct *work, unsigned long delay);

```

Листинг 28 — Инициализация очереди работ

Выделение и освобождение номеров устройств

```

extern int alloc_chrdev_region(dev_t *, unsigned, unsigned, const char *);
extern int register_chrdev_region(dev_t, unsigned, const char *);
extern int __register_chrdev(unsigned int major, unsigned int baseminor,
    unsigned int count, const char *name,
    const struct file_operations *fops);
extern void __unregister_chrdev(unsigned int major, unsigned int baseminor,
    unsigned int count, const char *name);
extern void unregister_chrdev_region(dev_t, unsigned);
extern void chrdev_show(struct seq_file *, off_t);

static inline int register_chrdev(unsigned int major, const char *name,
    const struct file_operations *fops)
{
    return __register_chrdev(major, 0, 256, name, fops);
}

static inline void unregister_chrdev(unsigned int major, const char *name)
{
    __unregister_chrdev(major, 0, 256, name);
}

MAJOR(dev_t dev); /* get major number */
MINOR(dev_t dev); /* get minor number */

```



```
/* Having major and minor numbers, they can be combined into dev_t: */
MKDEV(int major, int minor); /* combine into dev_t */
```

Листинг 29 — Выделение и освобождение номеров устройств

printk flags

```
#define KERN_EMERG KERN_SOH "0" /* system is unusable */
#define KERN_ALERT KERN_SOH "1" /* action must be taken immediately */
#define KERN_CRIT KERN_SOH "2" /* critical conditions */
#define KERN_ERR KERN_SOH "3" /* error conditions */
#define KERN_WARNING KERN_SOH "4" /* warning conditions */
#define KERN_NOTICE KERN_SOH "5" /* normal but significant condition */
#define KERN_INFO KERN_SOH "6" /* informational */
#define KERN_DEBUG KERN_SOH "7" /* debug-level messages */
}
```

Листинг 30 — printk flags

task_struct

```
struct task_struct {
#ifdef CONFIG_THREAD_INFO_IN_TASK
    struct thread_info thread_info;
#endif
    unsigned int __state;
    randomized_struct_fields_start

    void *stack;
    refcount_t usage;
    /* Per task flags (PF_*), defined further below: */
    unsigned int flags;
    unsigned int ptrace;

#ifdef CONFIG_MEM_ALLOC_PROFILING
    struct alloc_tag *alloc_tag;
#endif

    int on_cpu;
    struct __call_single_node wake_entry;
    unsigned int wakee_flips;
    unsigned long wakee_flip_decay_ts;
    struct task_struct *last_wakee;

    int recent_used_cpu;
    int wake_cpu;
    int on_rq;

    int prio;
    int static_prio;
    int normal_prio;
    ...
} __attribute__((aligned (64)));
```

Листинг 31 — task_struct

Загрузка модуля

```
#include <linux/module.h>
#include <linux/init.h>

static int __init md_init(void)
{
    printk("Module loaded!\n");
    return 0;
}

static int __init md_init(void)
{
    struct task_struct *task = &init_task;
    do {
        printk(KERN_INFO "Task: pid: %d, comm: %s, ppid: %d, state: %ld,
flags: %x, prio: %d, policy: %d, root: %s, pwd: %s\n",
            task->pid, task->comm, task->parent->pid,
            task->state, task->flags, task->prio, task->policy,
            task->fs->root.dentry->d_iname, task->fs->pwd.dentry-
>d_name.name);
    } while ((task = next_task(task)) != &init_task);
    printk(KERN_INFO "Current: pid: %d, comm: %s, ppid: %d, state: %ld,
flags: %x, prio: %d, policy: %d, root: %s, pwd: %s\n",
        current->pid, current->comm, current->parent->pid,
        current->state, current->flags, current->prio, current->policy,
        current->fs->root.dentry->d_iname, current->fs->pwd.dentry-
>d_name.name);
    return 0;
}

static void __exit md_exit(void)
{
    printk("Module unloaded\n");
}

module_init(md_init);
module_exit(md_exit);
}
```

Листинг 32 — Загрузка модуля

Мультиплексор

```
#define _GNU_SOURCE
#include <errno.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/epoll.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <sys/socket.h>
#include <sys/stat.h>
#include <time.h>
```



```

#include <unistd.h>

#define SERVER_PORT      9878
#define SERVER_BACKLOG   5
#define BUF_SIZE         52
#define MAX_EVENTS       64

#define OP_PRODUCER 'P'
#define OP_CONSUMER 'C'

#define SEM_KEY          200
#define SEM_STORAGE_EMPTY 0
#define SEM_STORAGE_FULL 1
#define SEM_LOCK         -1
#define SEM_UNLOCK       1

static char ring_buffer[BUF_SIZE];
static int write_idx = 0;
static int read_idx = 0;

static int semaphore_id;
static int listen_fd;
static int epoll_fd;
static int should_exit = 0;

struct sembuf stop_write = {SEM_STORAGE_EMPTY, SEM_LOCK, IPC_NOWAIT | SEM_UNDO};
struct sembuf start_read = {SEM_STORAGE_FULL, SEM_UNLOCK, SEM_UNDO};
struct sembuf stop_read = {SEM_STORAGE_FULL, SEM_LOCK, IPC_NOWAIT | SEM_UNDO};
struct sembuf start_write = {SEM_STORAGE_EMPTY, SEM_UNLOCK, SEM_UNDO};

void producer(int fd) {
    char letter;
    if (recv(fd, &letter, 1, 0) <= 0) {
        return;
    }
    if (semop(semaphore_id, &stop_write, 1) == -1) {
        send(fd, "WAIT", 5, 0);
        printf("[P %d] FULL, rejected '%c'\n", fd, letter);
        return;
    }
    ring_buffer[write_idx] = letter;
    write_idx = (write_idx + 1) % BUF_SIZE;
    semop(semaphore_id, &start_read, 1);
    send(fd, "OK", 3, 0);
    printf("[P %d] put '%c'\n", fd, letter);
}

void consumer(int fd) {
    if (semop(semaphore_id, &stop_read, 1) == -1) {
        send(fd, "WAIT", 5, 0);
        printf("[C %d] EMPTY\n", fd);
        return;
    }
    char resp[2];
    resp[0] = ring_buffer[read_idx];

```



```

    resp[1] = '\\0';
    read_idx = (read_idx + 1) % BUF_SIZE;
    semop(semaphore_id, &start_write, 1);
    send(fd, resp, 2, 0);
    printf("[C %d] got '%c'\\n", fd, resp[0]);
}

void client_service(int fd, struct timespec* time_start) {
    char op;
    if (recv(fd, &op, 1, 0) <= 0) {
        close(fd);
        return;
    }
    if (op == OP_PRODUCER) {
        producer(fd);
    } else if (op == OP_CONSUMER) {
        consumer(fd);
    }
    close(fd);
}

int main(void) {
    struct sockaddr_in server_addr = {0}, client_addr;
    socklen_t client_len = sizeof(client_addr);
    if ((listen_fd = socket(AF_INET, SOCK_STREAM, 0)) == -1) {
        perror("socket");
        exit(1);
    }
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(SERVER_PORT);
    if (bind(listen_fd, (struct sockaddr*)&server_addr,
sizeof(server_addr)) == -1) {
        perror("bind");
        exit(1);
    }
    if (listen(listen_fd, SERVER_BACKLOG) == -1) {
        perror("listen");
        exit(1);
    }
    int permissions = S_IRWXU | S_IRWXG | S_IRWXO;
    semaphore_id = semget(SEM_KEY, 2, IPC_CREAT | permissions);
    if (semaphore_id == -1) {
        perror("semget");
        exit(1);
    }
    if (semctl(semaphore_id, SEM_STORAGE_EMPTY, SETVAL, BUF_SIZE) == -1) {
        perror("semctl");
        exit(1);
    }
    if (semctl(semaphore_id, SEM_STORAGE_FULL, SETVAL, 0) == -1) {
        perror("semctl");
        exit(1);
    }
    epoll_fd = epoll_create1(0);
    if (epoll_fd == -1) {
        perror("epoll_create1");
    }
}

```



```

        exit(1);
    }
    struct epoll_event ev;
    ev.events = EPOLLIN;
    ev.data.fd = listen_fd;
    if (epoll_ctl(epoll_fd, EPOLL_CTL_ADD, listen_fd, &ev) == -1) {
        perror("epoll_ctl");
        exit(1);
    }
    struct epoll_event events[MAX_EVENTS];
    while (!should_exit) {
        int nfd = epoll_wait(epoll_fd, events, MAX_EVENTS, -1);
        if (nfd == -1) {
            perror("epoll_wait");
            break;
        }
        for (int i = 0; i < nfd; i++) {
            if (events[i].data.fd == listen_fd) {
                int conn_fd = accept(listen_fd, (struct
sockaddr*)&client_addr, &client_len);
                if (conn_fd == -1) {
                    perror("accept");
                    continue;
                }
                ev.events = EPOLLIN;
                ev.data.fd = conn_fd;
                if (epoll_ctl(epoll_fd, EPOLL_CTL_ADD, conn_fd, &ev) ==
-1) {
                    perror("epoll_ctl");
                    close(conn_fd);
                    continue;
                }
            } else {
                client_service(events[i].data.fd, &time_start);
            }
        }
    }
    close(epoll_fd);
    if (semctl(semaphore_id, 0, IPC_RMID) == -1) {
        perror("semctl");
        exit(1);
    }
    return EXIT_SUCCESS;
}

```

Листинг 33 — Сервер

```

#include <arpa/inet.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

```



```

#define SERVER_IP    "127.0.0.1"
#define SERVER_PORT  9878
#define MSG_SIZE     8

int main() {
    struct sockaddr_in server_addr, from_addr;
    srand(time(NULL));
    char type;
    char buf[64];
    socklen_t from_len = sizeof(from_addr);
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    inet_pton(AF_INET, SERVER_IP, &server_addr.sin_addr);
    for (int i = 0; i < 400; i++) {
        int sockfd = socket(AF_INET, SOCK_STREAM, 0);
        if (sockfd < 0) {
            perror("socket");
        } else if (connect(sockfd, (struct sockaddr *)&server_addr,
sizeof(server_addr)) < 0) {
            perror("connect");
            close(sockfd);
        } else {
            if (i % 3 == 0)
                type = 'p';
            else
                type = 'c';

            if (send(sockfd, &type, 1, 0) == -1) {
                perror("send");
            } else {
                if (type == 'p') {
                    int bytes_received = recvfrom(sockfd, buf,
sizeof(buf), 0, (struct sockaddr *)&from_addr, &from_len);
                    if (bytes_received == -1) {
                        perror("recvfrom");
                    } else if (strcmp(buf, "OK") == 0) {
                        printf("Client: Sent 'p', received : OK\n");
                    } else {
                        printf("Client: Sent 'p', received error\n");
                    }
                } else {
                    int bytes_received = recvfrom(sockfd, buf, sizeof(buf)
- 1, 0, (struct sockaddr *)&from_addr, &from_len);
                    if (bytes_received == -1) {
                        perror("recvfrom");
                    } else if (strcmp(buf, "OK") == 0) {
                        printf("Client: Sent 'c', received : OK\n");
                    } else {
                        printf("Client: Sent 'c', received error\n");
                    }
                }
            }
            close(sockfd);
        }
    }
}

```



```
    return 0;
}
```

Листинг 34 — Клиент

Многопоточный

```
#define PORT 9877
#define N 100
#define SB 0
#define SE 1
#define SF 2
#define P -1
#define V 1
struct sembuf start_produce[2] = { {SE, P, IPC_NOWAIT}, {SB, P, 0} };
struct sembuf stop_produce[2] = { {SB, V, 0}, {SF, V, 0} };
struct sembuf start_consume[2] = { {SF, P, IPC_NOWAIT}, {SB, P, 0} };
struct sembuf stop_consume[2] = { {SB, V, 0}, {SE, V, 0} };
int semfd;
char buffer[N];
int i, j;
char letter = 'a';
FILE *log_fp;
int flag = 1;
void sig_handler(int signum) {
    flag = 0;
    printf("catch sigint\n");
}
void* produce(void* arg) {
    struct timespec start, end;
    int fd = (int)(intptr_t)arg;
    clock_gettime(CLOCK_MONOTONIC, &start);
    if (semop(semfd, start_produce, 2) == -1) {
        if (errno == EAGAIN) {
            char err = 'e';
            if (send(fd, &err, 1, 0) < 0) {
                perror("send");
            }
        } else {
            perror("semop");
        }
        close(fd);
        return NULL;
    }
    buffer[i] = letter;
    printf("[Producer] Put '%c' at index %d\n", letter, i);
    i = (i + 1) % N;
    if (letter == 'z') {
        letter = 'a';
    } else {
        letter++;
    }
    if (semop(semfd, stop_produce, 2) == -1) {
        perror("semop\n");
        printf("errno : %d\n", errno);
        return NULL;
    }
}
```



```

    }
    if (send(fd, "OK", 2, 0) < 0) {
        perror("send");
        return NULL;
    }
    close(fd);
    clock_gettime(CLOCK_MONOTONIC, &end);
    long long elapsed = (end.tv_sec - start.tv_sec) * 1000000000LL +
(end.tv_nsec - start.tv_nsec);
    fprintf(log_fp, "%lld\n", elapsed);
    fflush(log_fp);
    return NULL;
}

void* consume(void* arg) {
    struct timespec start, end;
    int fd = (int)(intptr_t)arg;
    clock_gettime(CLOCK_MONOTONIC, &start);
    if (semop(semfd, start_consume, 2) == -1) {
        if (errno == EAGAIN) {
            char err = 'e';
            if (send(fd, &err, 1, 0) < 0) {
                perror("send");
            }
        } else {
            perror("semop");
        }
        close(fd);
        return NULL;
    }
    char item = buffer[j];
    printf("[Consumer] Got '%c' from index %d\n", item, j);
    j = (j + 1) % N;
    if (semop(semfd, stop_consume, 2) == -1) {
        perror("semop\n");
        printf("errno : %d\n", errno);
        return NULL;
    }
    if (send(fd, "OK", 2, 0) < 0) {
        perror("send");
        return NULL;
    }
    close(fd);
    clock_gettime(CLOCK_MONOTONIC, &end);
    long long elapsed = (end.tv_sec - start.tv_sec) * 1000000000LL +
(end.tv_nsec - start.tv_nsec);
    fprintf(log_fp, "%lld\n", elapsed);
    fflush(log_fp);
    return NULL;
}

int main() {
    struct sockaddr_in server_addr, client_addr;
    int listenfd;
    key_t semkey;
    pthread_attr_t attr;
    struct sigaction sa;
    int perms = S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH;

```



```

pthread_attr_init(&attr);
pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);
log_fp = fopen("measures.txt", "w");
if (!log_fp) {
    perror("fopen");
    exit(1);
}
if ((semkey = ftok("./ipc_key", 1)) == -1) {
    perror("ftok");
    exit(1);
}
if ((semfd = semget(semkey, 3, IPC_CREAT | perms)) == -1) {
    perror("semget");
    exit(1);
}
if (semctl(semfd, SB, SETVAL, 1) == -1) {
    perror("semctl\n");
    exit(1);
}
if (semctl(semfd, SE, SETVAL, N) == -1) {
    perror("semctl\n");
    exit(1);
}
if (semctl(semfd, SF, SETVAL, 0) == -1) {
    perror("semctl\n");
    exit(1);
}
listenfd = socket(AF_INET, SOCK_STREAM, 0);
if (listenfd < 0) {
    perror("socket");
    exit(1);
}
int opt = 1;
if (setsockopt(listenfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt))
< 0) {
    perror("setsockopt");
    exit(1);
}
server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = htonl(INADDR_ANY);
server_addr.sin_port = htons(PORT);
if (bind(listenfd, (struct sockaddr*)&server_addr,
sizeof(server_addr)) < 0) {
    perror("bind");
    exit(1);
}
if (listen(listenfd, 50) < 0) {
    perror("listen");
    exit(1);
}
sa.sa_handler = sig_handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;
if (sigaction(SIGINT, &sa, NULL) < 0) {
    perror("sigaction");
    exit(1);
}

```

```

1 // Файловый каталог
2 //
3 //
4 //
5 //
6 //
7 //
8 //
9 //
10 //
11 //
12 //
13 //
14 //
15 //
16 //
17 //
18 //
19 //
20 //
21 //
22 //
23 //
24 //
25 //
26 //
27 //
28 //
29 //
30 //
31 //
32 //
33 //
34 //
35 //
36 //
37 //
38 //
39 //
40 //
41 //
42 //
43 //
44 //
45 //
46 //
47 //
48 //
49 //
50 //
51 //
52 //
53 //
54 //
55 //
56 //
57 //
58 //
59 //
60 //
61 //
62 //
63 //
64 //
65 //
66 //
67 //
68 //
69 //
70 //
71 //
72 //
73 //
74 //
75 //
76 //
77 //
78 //
79 //
80 //
81 //
82 //
83 //
84 //
85 //
86 //
87 //
88 //
89 //
90 //
91 //
92 //
93 //
94 //
95 //
96 //
97 //
98 //
99 //
100 //

```

Диаграмм 66. Структурная функциональная организация на структурной схеме

```

1 // Файловый каталог
2 //
3 //
4 //
5 //
6 //
7 //
8 //
9 //
10 //
11 //
12 //
13 //
14 //
15 //
16 //
17 //
18 //
19 //
20 //
21 //
22 //
23 //
24 //
25 //
26 //
27 //
28 //
29 //
30 //
31 //
32 //
33 //
34 //
35 //
36 //
37 //
38 //
39 //
40 //
41 //
42 //
43 //
44 //
45 //
46 //
47 //
48 //
49 //
50 //
51 //
52 //
53 //
54 //
55 //
56 //
57 //
58 //
59 //
60 //
61 //
62 //
63 //
64 //
65 //
66 //
67 //
68 //
69 //
70 //
71 //
72 //
73 //
74 //
75 //
76 //
77 //
78 //
79 //
80 //
81 //
82 //
83 //
84 //
85 //
86 //
87 //
88 //
89 //
90 //
91 //
92 //
93 //
94 //
95 //
96 //
97 //
98 //
99 //
100 //

```

202 / 289

13.5 Структурная функциональная организация на структурной схеме

13.5 Структурная функциональная организация на структурной схеме

13.5 Структурная функциональная организация на структурной схеме

207 / 289

```

1 // Файловый каталог
2 //
3 //
4 //
5 //
6 //
7 //
8 //
9 //
10 //
11 //
12 //
13 //
14 //
15 //
16 //
17 //
18 //
19 //
20 //
21 //
22 //
23 //
24 //
25 //
26 //
27 //
28 //
29 //
30 //
31 //
32 //
33 //
34 //
35 //
36 //
37 //
38 //
39 //
40 //
41 //
42 //
43 //
44 //
45 //
46 //
47 //
48 //
49 //
50 //
51 //
52 //
53 //
54 //
55 //
56 //
57 //
58 //
59 //
60 //
61 //
62 //
63 //
64 //
65 //
66 //
67 //
68 //
69 //
70 //
71 //
72 //
73 //
74 //
75 //
76 //
77 //
78 //
79 //
80 //
81 //
82 //
83 //
84 //
85 //
86 //
87 //
88 //
89 //
90 //
91 //
92 //
93 //
94 //
95 //
96 //
97 //
98 //
99 //
100 //

```

Диаграмм 67. Структурная функциональная организация на структурной схеме

203 / 289

13.6 Структурная функциональная организация на структурной схеме

13.6 Структурная функциональная организация на структурной схеме

13.6 Структурная функциональная организация на структурной схеме

13.6 Структурная функциональная организация на структурной схеме

208 / 289

13

Описание

Внешнее устройство, которое используется для хранения данных (файлов) и их обработки.

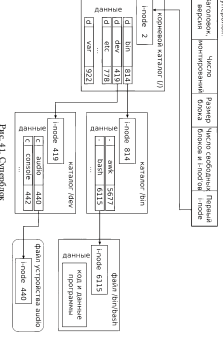


Рис. 13. Структурная функциональная организация на структурной схеме

209 / 289

13.6 Структурная функциональная организация на структурной схеме

13.6 Структурная функциональная организация на структурной схеме

13.6 Структурная функциональная организация на структурной схеме

13.6 Структурная функциональная организация на структурной схеме

13.6 Структурная функциональная организация на структурной схеме

13.6 Структурная функциональная организация на структурной схеме

13.6 Структурная функциональная организация на структурной схеме

13.6 Структурная функциональная организация на структурной схеме

210 / 289



Символьный уровень — это уровень, на котором файлы и каталоги представляются в виде символов.

13.6 Структурная функциональная организация на структурной схеме

206 / 289

13.6 Структурная функциональная организация на структурной схеме

13.6 Структурная функциональная организация на структурной схеме

13.6 Структурная функциональная организация на структурной схеме

13.6 Структурная функциональная организация на структурной схеме

211 / 289

```

    }
    while (flag) {
        socklen_t clilen = sizeof(client_addr);
        int connfd = accept(listenfd, (struct sockaddr*)&client_addr,
&clilen);
        if (connfd < 0) {
            perror("accept");
            if (errno == EINTR)
                exit(1);
        }
        char type;
        if (recv(connfd, &type, 1, 0) <= 0) {
            close(connfd);
            exit(1);
        }

        pthread_t tid;
        if (type == 'p') {
            pthread_create(&tid, &attr, produce, (void*)(intptr_t)connfd);
        } else if (type == 'c') {
            pthread_create(&tid, &attr, consume, (void*)(intptr_t)connfd);
        } else {
            close(connfd);
        }
    }
    close(listenfd);
    if (fclose(log_fp) != 0) {
        perror("fclose");
        exit(1);
    }
    if (semctl(semfd, 0, IPC_RMID) < 0) {
        perror("semctl");
        exit(1);
    }
    return 0;
}

```

Листинг 35 — Сервер

Передача данных из АП ядра в АП пользователя и обратно (фортуны)

```

#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/proc_fs.h>
#include <linux/uaccess.h>
#include <linux/vmalloc.h>
#include <linux/string.h>
#include <linux/errno.h>

#define PROC_FILE_NAME "fortune"
#define PROC_DIR_NAME "fortune_dir"
#define PROC_SYM_NAME "fortune_symlink"

#define COOKIE_BUF_SIZE PAGE_SIZE

```



```

static char *cookie_buf;
static unsigned int write_index;
static unsigned int read_index;
static struct proc_dir_entry *proc_file;
static struct proc_dir_entry *proc_dir;
static struct proc_dir_entry *proc_symlink_entry;

static ssize_t fortune_write(struct file *file, const char __user
*user_buf, size_t count, loff_t *ppos) {
    size_t len;

    if (count == 0) return 0;
    if (count > COOKIE_BUF_SIZE - 1) return -ENOSPC;
    if (write_index + count + 1 > COOKIE_BUF_SIZE) return -ENOSPC;
    len = count;
    if (copy_from_user(cookie_buf + write_index, user_buf, len)) {
        return -EFAULT;
    }
    cookie_buf[write_index + len] = '\\0';
    printk(KERN_INFO "fortune: write %s", cookie_buf + write_index);
    write_index += len + 1;
    return len;
}

static ssize_t fortune_read(struct file *file, char __user *user_buf,
size_t count, loff_t *ppos) {
    size_t len;
    size_t available;

    if (*ppos > 0) return 0;
    if (count == 0) return 0;
    if (write_index == 0) return 0;
    if (read_index >= write_index) read_index = 0;
    available = write_index - read_index;
    len = strlen(cookie_buf + read_index, available);
    if (len == 0) {
        read_index = 0;
        len = strlen(cookie_buf, write_index);
        if (len == 0) return 0;
    }
    if (len > count) len = count;
    if (copy_to_user(user_buf, cookie_buf + read_index, len)) {
        return -EFAULT;
    }
    printk(KERN_INFO "fortune: read %s", cookie_buf + read_index);
    read_index += len + 1;
    if (read_index >= write_index)
        read_index = 0;
    *ppos += len;
    return len;
}

static int fortune_open(struct inode *inode, struct file *file) {
    printk(KERN_INFO "fortune: open");
    return 0;
}

```



```

static int fortune_release(struct inode *inode, struct file *file) {
    printk(KERN_INFO "fortune: release");
    return 0;
}

static const struct proc_ops fortune_proc_ops = {
    .proc_open = fortune_open,
    .proc_read = fortune_read,
    .proc_write = fortune_write,
    .proc_release = fortune_release,
};

static int __init fortune_init(void) {
    cookie_buf = vmalloc(COOKIE_BUF_SIZE);
    if (!cookie_buf) {
        printk(KERN_ERR "fortune: cannot allocate cookie buffer");
        return -ENOMEM;
    }
    memset(cookie_buf, 0, COOKIE_BUF_SIZE);
    write_index = 0;
    read_index = 0;
    proc_dir = proc_mkdir(PROC_DIR_NAME, NULL);
    if (!proc_dir) {
        printk(KERN_ERR "fortune: cannot create /proc/%s", PROC_DIR_NAME);
        vfree(cookie_buf);
        return -ENOMEM;
    }
    proc_file = proc_create(PROC_FILE_NAME, 0666, proc_dir,
&fortune_proc_ops);
    if (!proc_file) {
        printk(KERN_ERR "fortune: cannot create /proc/%s",
PROC_FILE_NAME);
        proc_remove(proc_dir);
        vfree(cookie_buf);
        return -ENOMEM;
    }
    proc_symlink_entry = proc_symlink(PROC_SYM_NAME, NULL, "/proc/"
PROC_DIR_NAME "/" PROC_FILE_NAME);
    if (!proc_symlink_entry) {
        printk(KERN_ERR "fortune: cannot create /proc/%s", PROC_SYM_NAME);
        proc_remove(proc_dir);
        proc_remove(proc_file);
        vfree(cookie_buf);
        return -ENOMEM;
    }
    printk(KERN_INFO "fortune: module loaded");
    return 0;
}

static void __exit fortune_exit(void) {
    proc_remove(proc_symlink_entry);
    proc_remove(proc_file);
    proc_remove(proc_dir);
    vfree(cookie_buf);
    printk(KERN_INFO "fortune: module unloaded");
}

```



```

}

module_init(fortune_init);
module_exit(fortune_exit);

```

Листинг 36 — Фортунки

Монтирование ФС

```

#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/fs.h>
#include <asm/atomic.h>
#include <asm/uaccess.h>

#define MYFS_MAGIC_NUMBER 0x77777777
static void myfs_put_super(struct super_block *sb) {
    printk("myfs: put_super\n");
}
static struct super_operations const myfs_super_ops = {
    .put_super = myfs_put_super,
    .statfs = simple_statfs,
    .drop_inode = generic_delete_inode
};
static int myfs_fill_sb(struct super_block *sb, void *data, int silent) {
    struct inode *inode;
    struct dentry *root;
    sb->s_blocksize = PAGE_SIZE;
    sb->s_blocksize_bits = PAGE_SHIFT;
    sb->s_magic = MYFS_MAGIC_NUMBER;
    sb->s_op = &myfs_super_ops;
    sb->s_time_gran = 1;
    inode = new_inode(sb);
    if (!inode) {
        printk(KERN_ERR "myfs: new_inode error\n");
        return -ENOMEM;
    }
    inode->i_ino = 1;
    inode->i_mode = S_IFDIR | 0755;
    simple_inode_init_ts(inode);

    inode->i_op = &simple_dir_inode_operations;
    inode->i_fop = &simple_dir_operations;

    root = d_make_root(inode);
    printk("myfs: d_make_root\n");
    if (!root) {
        printk(KERN_ERR "myfs: d_make_root error\n");
        iput(inode);
        return -ENOMEM;
    }
    sb->s_root = root;
    printk(KERN_INFO "myfs: VFS root created\n");
    printk(KERN_INFO "myfs: super_block init");
    return 0;
}

```

Плане 75 – информация о фалитонной системе процесса; в листине через него получают имя корня фалитонной системы ($\text{f} \rightarrow \text{f} \cdot \text{root} \cdot \text{dent} \text{tu}$) и имя текущего фалитонного катализатора ($\text{f} \rightarrow \text{f} \cdot \text{root} \cdot \text{dent} \text{tu}$).

```

}
static struct dentry *myfs_mount(struct file_system_type *type, int flags,
const char *dev, void *data) {
    struct dentry *const entry = mount_nodev(type, flags, data,
myfs_fill_sb);
    if (IS_ERR(entry))
        printk(KERN_ERR "myfs: mount error\n");
    else
        printk(KERN_INFO "myfs: mount\n");
    return entry;
}
static struct file_system_type myfs_type = {
    .owner    = THIS_MODULE,
    .name     = "myfs",
    .mount    = myfs_mount,
    .kill_sb  = kill_litter_super
};
static int __init myfs_init(void) {
    int ret;

    printk(KERN_INFO "myfs: register_filesystem call\n");
    ret = register_filesystem(&myfs_type);
    if (ret) {
        printk(KERN_ERR "myfs: register_filesystem error\n");
        return ret;
    }
    printk(KERN_INFO "myfs: loaded\n");
    return 0;
}
static void __exit myfs_exit(void) {
    int ret;
    ret = unregister_filesystem(&myfs_type);
    if (ret)
        printk(KERN_ERR "myfs: unregister_filesystem error\n");
    else
        printk("myfs: unregister_filesystem\n");
    printk(KERN_INFO "myfs: unloaded\n");
}
module_init(myfs_init);
module_exit(myfs_exit);

```

Листинг 37 — Монтирование

Slab кэш

```

struct myfs_inode {
    int i_mode;
    unsigned long i_ino;
};

static struct kmem_cache* inode_cache = NULL;
static struct myfs_inode** inode_cached_array = NULL;

static struct dentry *my_fs_mount(struct file_system_type *type, int
flags,
                                const char *dev, void *data)

```

```

12  var count = 0;
13  if (count < 10) {
14    if (count % 2 == 0) {
15      console.log('even', count + ' is even');
16    } else {
17      console.log('odd', count + ' is odd');
18    }
19    count++;
20  }
21  // 0 is even, 1 is odd, 2 is even, 3 is odd, 4 is even, 5 is odd, 6 is even, 7 is odd, 8 is even, 9 is odd
22  return count;
23 }
24
25 // 10 is even, 11 is odd, 12 is even, 13 is odd, 14 is even, 15 is odd, 16 is even, 17 is odd, 18 is even, 19 is odd
26 // 20 is even, 21 is odd, 22 is even, 23 is odd, 24 is even, 25 is odd, 26 is even, 27 is odd, 28 is even, 29 is odd
27
28 // 30 is even, 31 is odd, 32 is even, 33 is odd, 34 is even, 35 is odd, 36 is even, 37 is odd, 38 is even, 39 is odd
29
30 // 40 is even, 41 is odd, 42 is even, 43 is odd, 44 is even, 45 is odd, 46 is even, 47 is odd, 48 is even, 49 is odd
31
32 // 50 is even, 51 is odd, 52 is even, 53 is odd, 54 is even, 55 is odd, 56 is even, 57 is odd, 58 is even, 59 is odd
33
34 // 60 is even, 61 is odd, 62 is even, 63 is odd, 64 is even, 65 is odd, 66 is even, 67 is odd, 68 is even, 69 is odd
35
36 // 70 is even, 71 is odd, 72 is even, 73 is odd, 74 is even, 75 is odd, 76 is even, 77 is odd, 78 is even, 79 is odd
37
38 // 80 is even, 81 is odd, 82 is even, 83 is odd, 84 is even, 85 is odd, 86 is even, 87 is odd, 88 is even, 89 is odd
39
40 // 90 is even, 91 is odd, 92 is even, 93 is odd, 94 is even, 95 is odd, 96 is even, 97 is odd, 98 is even, 99 is odd
41
42 // 100 is even, 101 is odd, 102 is even, 103 is odd, 104 is even, 105 is odd, 106 is even, 107 is odd, 108 is even, 109 is odd
43
44 // 110 is even, 111 is odd, 112 is even, 113 is odd, 114 is even, 115 is odd, 116 is even, 117 is odd, 118 is even, 119 is odd
45
46 // 120 is even, 121 is odd, 122 is even, 123 is odd, 124 is even, 125 is odd, 126 is even, 127 is odd, 128 is even, 129 is odd
47
48 // 130 is even, 131 is odd, 132 is even, 133 is odd, 134 is even, 135 is odd, 136 is even, 137 is odd, 138 is even, 139 is odd
49
50 // 140 is even, 141 is odd, 142 is even, 143 is odd, 144 is even, 145 is odd, 146 is even, 147 is odd, 148 is even, 149 is odd
51
52 // 150 is even, 151 is odd, 152 is even, 153 is odd, 154 is even, 155 is odd, 156 is even, 157 is odd, 158 is even, 159 is odd
53
54 // 160 is even, 161 is odd, 162 is even, 163 is odd, 164 is even, 165 is odd, 166 is even, 167 is odd, 168 is even, 169 is odd
55
56 // 170 is even, 171 is odd, 172 is even, 173 is odd, 174 is even, 175 is odd, 176 is even, 177 is odd, 178 is even, 179 is odd
57
58 // 180 is even, 181 is odd, 182 is even, 183 is odd, 184 is even, 185 is odd, 186 is even, 187 is odd, 188 is even, 189 is odd
59
60 // 190 is even, 191 is odd, 192 is even, 193 is odd, 194 is even, 195 is odd, 196 is even, 197 is odd, 198 is even, 199 is odd
61
62 // 200 is even, 201 is odd, 202 is even, 203 is odd, 204 is even, 205 is odd, 206 is even, 207 is odd, 208 is even, 209 is odd
63
64 // 210 is even, 211 is odd, 212 is even, 213 is odd, 214 is even, 215 is odd, 216 is even, 217 is odd, 218 is even, 219 is odd
65
66 // 220 is even, 221 is odd, 222 is even, 223 is odd, 224 is even, 225 is odd, 226 is even, 227 is odd, 228 is even, 229 is odd
67
68 // 230 is even, 231 is odd, 232 is even, 233 is odd, 234 is even, 235 is odd, 236 is even, 237 is odd, 238 is even, 239 is odd
69
70 // 240 is even, 241 is odd, 242 is even, 243 is odd, 244 is even, 245 is odd, 246 is even, 247 is odd, 248 is even, 249 is odd
71
72 // 250 is even, 251 is odd, 252 is even, 253 is odd, 254 is even, 255 is odd, 256 is even, 257 is odd, 258 is even, 259 is odd
73
74 // 260 is even, 261 is odd, 262 is even, 263 is odd, 264 is even, 265 is odd, 266 is even, 267 is odd, 268 is even, 269 is odd
75
76 // 270 is even, 271 is odd, 272 is even, 273 is odd, 274 is even, 275 is odd, 276 is even, 277 is odd, 278 is even, 279 is odd
77
78 // 280 is even, 281 is odd, 282 is even, 283 is odd, 284 is even, 285 is odd, 286 is even, 287 is odd, 288 is even, 289 is odd
79
80 // 290 is even, 291 is odd, 292 is even, 293 is odd, 294 is even, 295 is odd, 296 is even, 297 is odd, 298 is even, 299 is odd
81
82 // 300 is even, 301 is odd, 302 is even, 303 is odd, 304 is even, 305 is odd, 306 is even, 307 is odd, 308 is even, 309 is odd
83
84 // 310 is even, 311 is odd, 312 is even, 313 is odd, 314 is even, 315 is odd, 316 is even, 317 is odd, 318 is even, 319 is odd
85
86 // 320 is even, 321 is odd, 322 is even, 323 is odd, 324 is even, 325 is odd, 326 is even, 327 is odd, 328 is even, 329 is odd
87
88 // 330 is even, 331 is odd, 332 is even, 333 is odd, 334 is even, 335 is odd, 336 is even, 337 is odd, 338 is even, 339 is odd
89
90 // 340 is even, 341 is odd, 342 is even, 343 is odd, 344 is even, 345 is odd, 346 is even, 347 is odd, 348 is even, 349 is odd
91
92 // 350 is even, 351 is odd, 352 is even, 353 is odd, 354 is even, 355 is odd, 356 is even, 357 is odd, 358 is even, 359 is odd
93
94 // 360 is even, 361 is odd, 362 is even, 363 is odd, 364 is even, 365 is odd, 366 is even, 367 is odd, 368 is even, 369 is odd
95
96 // 370 is even, 371 is odd, 372 is even, 373 is odd, 374 is even, 375 is odd, 376 is even, 377 is odd, 378 is even, 379 is odd
97
98 // 380 is even, 381 is odd, 382 is even, 383 is odd, 384 is even, 385 is odd, 386 is even, 387 is odd, 388 is even, 389 is odd
99
100 // 390 is even, 391 is odd, 392 is even, 393 is odd, 394 is even, 395 is odd, 396 is even, 397 is odd, 398 is even, 399 is odd
101
102 // 400 is even, 401 is odd, 402 is even, 403 is odd, 404 is even, 405 is odd, 406 is even, 407 is odd, 408 is even, 409 is odd
103
104 // 410 is even, 411 is odd, 412 is even, 413 is odd, 414 is even, 415 is odd, 416 is even, 417 is odd, 418 is even, 419 is odd
105
106 // 420 is even, 421 is odd, 422 is even, 423 is odd, 424 is even, 425 is odd, 426 is even, 427 is odd, 428 is even, 429 is odd
107
108 // 430 is even, 431 is odd, 432 is even, 433 is odd, 434 is even, 435 is odd, 436 is even, 437 is odd, 438 is even, 439 is odd
109
110 // 440 is even, 441 is odd, 442 is even, 443 is odd, 444 is even, 445 is odd, 446 is even, 447 is odd, 448 is even, 449 is odd
111
112 // 450 is even, 451 is odd, 452 is even, 453 is odd, 454 is even, 455 is odd, 456 is even, 457 is odd, 458 is even, 459 is odd
113
114 // 460 is even, 461 is odd, 462 is even, 463 is odd, 464 is even, 465 is odd, 466 is even, 467 is odd, 468 is even, 469 is odd
115
116 // 470 is even, 471 is odd, 472 is even, 473 is odd, 474 is even, 475 is odd, 476 is even, 477 is odd, 478 is even, 479 is odd
117
118 // 480 is even, 481 is odd, 482 is even, 483 is odd, 484 is even, 485 is odd, 486 is even, 487 is odd, 488 is even, 489 is odd
119
120 // 490 is even, 491 is odd, 492 is even, 493 is odd, 494 is even, 495 is odd, 496 is even, 497 is odd, 498 is even, 499 is odd
121
122 // 500 is even, 501 is odd, 502 is even, 503 is odd, 504 is even, 505 is odd, 506 is even, 507 is odd, 508 is even, 509 is odd
123
124 // 510 is even, 511 is odd, 512 is even, 513 is odd, 514 is even, 515 is odd, 516 is even, 517 is odd, 518 is even, 519 is odd
125
126 // 520 is even, 521 is odd, 522 is even, 523 is odd, 524 is even, 525 is odd, 526 is even, 527 is odd, 528 is even, 529 is odd
127
128 // 530 is even, 531 is odd, 532 is even, 533 is odd, 534 is even, 535 is odd
```

252 / 289

[illegible]

253 / 289

```

120 static void addT(Fortune, addTNode) {
121   gNode = new gNode(gNode, addTNode);
122   gNode->prevNode = gNode;
123   gNode->nextNode = gNode;
124   vNode->prevNode = gNode;
125   vNode->nextNode = gNode;
126 }
127
128 module, listT(Fortune, listT);
129 module, addT(Fortune, addT);

```

254 / 289

[illegible]

255 / 289

Вопросы

III

СОДЕРЖАНИЕ

1	ГЛАВНОЕ
11	Введение
12	История создания
13	Цели и задачи
14	Методология
15	Результаты исследования
16	Выводы
17	Список литературы
18	Приложение
19	Справочные материалы
20	Словарь терминов
21	Список сокращений
22	Список использованных источников
23	Список использованных документов
24	Список использованных программ
25	Список использованных баз данных
26	Список использованных карт
27	Список использованных таблиц
28	Список использованных рисунков
29	Список использованных фотографий
30	Список использованных видеозаписей
31	Список использованных аудиозаписей
32	Список использованных документов
33	Список использованных программ
34	Список использованных баз данных
35	Список использованных карт
36	Список использованных таблиц
37	Список использованных рисунков
38	Список использованных фотографий
39	Список использованных видеозаписей
40	Список использованных аудиозаписей
41	Список использованных документов
42	Список использованных программ
43	Список использованных баз данных
44	Список использованных карт
45	Список использованных таблиц
46	Список использованных рисунков
47	Список использованных фотографий
48	Список использованных видеозаписей
49	Список использованных аудиозаписей
50	Список использованных документов
51	Список использованных программ
52	Список использованных баз данных
53	Список использованных карт
54	Список использованных таблиц
55	Список использованных рисунков
56	Список использованных фотографий
57	Список использованных видеозаписей
58	Список использованных аудиозаписей
59	Список использованных документов
60	Список использованных программ
61	Список использованных баз данных
62	Список использованных карт
63	Список использованных таблиц
64	Список использованных рисунков
65	Список использованных фотографий
66	Список использованных видеозаписей
67	Список использованных аудиозаписей
68	Список использованных документов
69	Список использованных программ
70	Список использованных баз данных
71	Список использованных карт
72	Список использованных таблиц
73	Список использованных рисунков
74	Список использованных фотографий
75	Список использованных видеозаписей
76	Список использованных аудиозаписей
77	Список использованных документов
78	Список использованных программ
79	Список использованных баз данных
80	Список использованных карт
81	Список использованных таблиц
82	Список использованных рисунков
83	Список использованных фотографий
84	Список использованных видеозаписей
85	Список использованных аудиозаписей
86	Список использованных документов
87	Список использованных программ
88	Список использованных баз данных
89	Список использованных карт
90	Список использованных таблиц
91	Список использованных рисунков
92	Список использованных фотографий
93	Список использованных видеозаписей
94	Список использованных аудиозаписей
95	Список использованных документов
96	Список использованных программ
97	Список использованных баз данных
98	Список использованных карт
99	Список использованных таблиц
100	Список использованных рисунков

2 / 30

Ключевые слова

```

{
    ...
    for (i = 0; i < OBJECTS_PER_MOUNT; i++)
    {
        struct myfs_inode *inode = kmem_cache_alloc(inode_cache,
GFP_KERNEL);
        if (!inode)
        {
            for (int j = 0; j < i; j++)
                kmem_cache_free(inode_cache, inode_cached_array[j]);
            return ERR_PTR(-ENOMEM);
        }
        ...
    }
    ...
}

static int __init myfs_init(void) {
    ...
    // Создание slab-кэша в init
    inode_cache = kmem_cache_create(SLAB_NAME, sizeof(struct myfs_inode),
0, 0, NULL);
    if (inode_cache == NULL)
        return -ENOMEM;
    ...
}

static void __exit myfs_exit(void) {
    ...
    // освобождение всех объектов
    for (i = 0; i < cached_count; i++)
        kmem_cache_free(inode_cache, inode_cached_array[i]);
    // удаление slab-кэша
    kmem_cache_destroy(inode_cache);
    ...
}

```

Листинг 38 — Slab кэш

Тасклеты

```

static struct tasklet_struct *tasklet = NULL;
static irqreturn_t interrupt_handler(int irq, void *dev_id) {
    if (irq == IRQ_NO) {
        start_time = ktime_get();
        tasklet_schedule(tasklet);
        return IRQ_HANDLED;
    }
    return IRQ_NONE;
}

static int __init my_init(void) {
    int ret = request_irq(IRQ_NO, interrupt_handler, IRQF_SHARED,
        "interrupt_handler_tasklet", (void *)
(interrupt_handler));
    if (ret) return ret;
}

```



```

    tasklet = kcalloc(sizeof(struct tasklet_struct), GFP_KERNEL);
    if (!tasklet) {
        free_irq(IRQ_NO, (void *)(interrupt_handler));
        return -ENOMEM;
    }
    tasklet_init(tasklet, my_tasklet_fun, 0);
    return 0;
}

static void __exit my_exit(void) {
    tasklet_kill(tasklet);
    kfree(tasklet);
    free_irq(IRQ_NO, (void *)(interrupt_handler));
}

```

Листинг 39 — Тасклеты

Очередь работ

```

static irqreturn_t irq_handler(int irq, void *dev_id) {
    if (irq == IRQ_NO) {
        start_time = ktime_get();
        tasklet_schedule(tasklet);
        return IRQ_HANDLED;
    }
    return IRQ_NONE;
}

static int __init my_init(void)
{
    int ret;
    wq = alloc_workqueue("irq_wq", WQ_MEM_RECLAIM, 2);
    if (!wq)
        return -ENOMEM;
    work1 = kcalloc(sizeof(*work1), GFP_KERNEL);
    if (!work1) {
        destroy_workqueue(wq);
        return -ENOMEM;
    }
    work2 = kcalloc(sizeof(*work2), GFP_KERNEL);
    if (!work2) {
        destroy_workqueue(wq);
        kfree(work1);
        return -ENOMEM;
    }
    INIT_WORK(&work1->work, work1_fn);
    INIT_WORK(&work2->work, work2_fn);
    ret = request_irq(IRQ_NUM, irq_handler, IRQF_SHARED, "irq_workqueue",
    irq_handler);
    if (ret) {
        kfree(work1);
        kfree(work2);
        destroy_workqueue(wq);
        return ret;
    }
}

```



```

if (!proc_create(PROC_NAME, 0444, NULL, &proc_fops)) {
    free_irq(IRQ_NUM, irq_handler);
    kfree(work1);
    kfree(work2);
    destroy_workqueue(wq);
    return -ENOMEM;
}
printk(KERN_INFO "irq_workqueue: loaded\n");
return 0;
}

static void __exit my_exit(void) {
    remove_proc_entry(PROC_NAME, NULL);
    synchronize_irq(IRQ_NUM);
    free_irq(IRQ_NUM, irq_handler);
    flush_workqueue(wq);
    destroy_workqueue(wq);
    kfree(work1);
    kfree(work2);
    printk(KERN_INFO "irq_workqueue: unloaded\n");
}

```

Листинг 40 — Очередь работ

Ввод вывод

```

int main() {
    int fd1 = open("test.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    int fd2 = open("test.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    const char *str1 = "aaaaaaaaaaaa";
    write(fd1, str1, strlen(str1));
    const char *str2 = "BBBB";
    write(fd2, str2, strlen(str2));
    close(fd1);
    close(fd2);
    return 0;
}

```

Листинг 41 — Ввод вывод

